

Studentu informacine sistema
v2.0

Sugeneruota Doxygen 1.17.0

skyrius 1

Studentų Informacinė Sistema

OOP_Marius_Augustinas · VU ISI Objektinio programavimo kurso laboratoriniai darbai

C++ programa studentų duomenims įvesti, saugoti, apdoroti ir analizuoti. Palaiko tris STL kontenerius, tris duomenų įvesties būdus, studentų skirstymą į kategorijas ir spartos tyrimus su dideliais duomenų kiekiais.

1.1 Turinys

- 1. Versijų istorijaVersijų istorija
- 1. Naudojimosi instrukcijaNaudojimosi instrukcija
- 1. Įdiegimo instrukcijaĮdiegimo instrukcija
- 1. Klasų architektūraKlasų architektūra
- 1. DokumentacijaDokumentacija
- 1. Unit testaiUnit testai
- 1. Sartos tyrimaiSartos tyrimai
- 1. Failų struktūraFailų struktūra

1.2 1. Versijų istorija

Versija	Pagrindiniai pakeitimai
v0.↵ 1	Pradinė realizacija — <code>std::vector</code> , rankinis įvedimas, vidurkio arba medianos skaičiavimas
v0.↵ 2	Failo skaitymas ir rašymas, rikiavimas, abu galutiniai balai skaičiuojami vienu metu
v0.↵ 3	Kodas suskaidytas į modulius (<code>*.h/*.cpp</code>), išimčių valdymas (<code>try/catch</code>)
v0.↵ 4	Studentų skirstymas į dvi grupes, spartos testavimas, failų generatorius
v0.↵ 5	Programa pertvarkyta į <code>template<typename Container> — vector/list/deque</code>

Versija	Pagrindiniai pakeitimai
v1.↔ 0	Trijų skirstymo strategijų palyginimas, atminties analizė, CMakeLists.txt
v1.↔ 1	struct studentas → class studentas; I/O optimizacija; struct vs class tyrimas
v1.↔ 2	Pilna Rule of Five, perdengti operatoriai, rankinis klasės metodų testas
v1.↔ 5	Abstrakti bazinė klasė zmogus + išvestinė studentas; polimorfizmas
v2.↔ 0	Doxygen dokumentacija (HTML + LaTeX + PDF), Catch2 unit testai , švari repozitorija

1.3 2. Naudojimosi instrukcija

1.3.1 Paleidimas

```
.\programa.exe      # Windows
./programa          # Linux/Mac
```

1.3.2 Pirmas ekranas — konteinerio pasirinkimas

Pasirinkite konteinerį:

1. vector
2. list
3. deque

Pasirinkimas galioja visai sesijai. Norint pakeisti — paleisti programą iš naujo.

1.3.3 Meniu

Nr.	Veiksmas	Ką daro
1	Įvesti studentus ranka	Klausia vardo, pavardės, pažymių po vieną
2	Generuoti tik pažymius	Vardai įvedami ranka, pažymiai atsitiktiniai
3	Generuoti studentus	Šabloniniai vardai (VardasNR1) ir atsitiktiniai pažymiai
4	Nuskaityti iš failo	Klausia failo pavadinimo su .txt plėtiniu
5	Generuoti studentų failą	Sukuria testinį failą su nurodytu įrašų skaičiumi
6	Failų kūrimo testas	1 tyrimas — matuoja failo rašymo spartą
7	Duomenų apdorojimo testas	2 tyrimas — matuoja visus apdorojimo žingsnius
8	Konteinerių palyginimas	3 tyrimas — lygina vector, list, deque
9	Klasių metodų testas	Rankinis zmogus ir studentas metodų patikrinimas
0	Baigti darbą	

1.3.4 Tipinė darbo eiga

1. Paleisti programą → pasirinkti konteinerį (1 = vector)
2. Meniu punktas 5 → sugeneruoti failą (pvz. 100000 įrašų)
3. Meniu punktas 4 → nuskaityti tą failą
4. Pasirinkti rikiavimo būdą → pvz. 4 (pagal medianą)
5. Pasirinkti išvesties formatą → 1 (konsolė) arba 2 (failas)

Rezultate studentai suskirstomi į dvi grupes: **kietiakai** (galutinis 5.0) ir **vargsiukai** (galutinis < 5.0).

1.3.5 Duomenų failo formatas

Vardas	Pavarde	ND1	ND2	...	Egzaminas
VardasNR1	PavardeNR1	7	3	...	5
VardasNR2	PavardeNR2	9	1	...	8

Antraštė naudojama automatiškai nustatyti namų darbų stulpelių skaičių pagal ND žymėjimą — todėl failai su skirtingu pažymių kiekiu nuskaitomi be jokių pakeitimų kode.

1.3.6 Įvesties validacija

Tikrinama	Reakcija
Vardas / pavardė tuščia arba su skaičiais	Kartojamas klausimas
Pažymys ne tarp 1 ir 10	Kartojamas klausimas (interaktyviai) arba <code>std::runtime_error</code> (iš failo)
Failo pavadinimas be <code>.txt</code>	Kartojamas klausimas
Failas neegzistuoja	<code>std::runtime_error</code> su failo pavadinimu
Menu pasirinkimas už ribų	Kartojamas klausimas

1.4 3. Įdiegimo instrukcija

1.4.1 Reikalavimai

Įrankis	Versija	Būtinai
C++ kompiliatorius	MSVC 2019+, g++ 8+, clang++ 7+ (C++17)	Taip
CMake	3.16+	Rekomenduojama
Catch2	v2.13.x (<code>catch.hpp</code>)	Unit testams
Doxygen	1.9+	Dokumentacijai
Graphviz	bet kokia	Diagramoms dokumentacijoje
TeX Live / MiKTeX	bet kokia	PDF generavimui

1.4.2 Variantas A — CMake (rekomenduojama)

```
git clone <repozitorijos-adresas>
cd OOP_Marius_Augustinas
```

```
mkdir build && cd build
cmake ..
cmake --build . --config Release
```

Rezultatas:

- programa (arba `programa.exe`) — pagrindinė programa
- `unitTests` — unit testai (jei rastas `tests/catch.hpp`)
- `data/` aplankas nukopijuojamas automatiškai šalia vykdomojo failo

1.4.3 Variantas B — g++ tiesiogiai

```
cd OOP_2
g++ -std=c++17 -O2 -Wall \
    OOP_2.cpp zmogus.cpp studentas.cpp testStudentas.cpp \
    menu.cpp enter.cpp generate.cpp file.cpp print.cpp test.cpp \
    -o programa.exe
./programa.exe
```

Kompiliuoti reikia iš **OOP_2/ aplanko** — taip vykdomasis failas atsiranda šalia `data/`, ir testavimo funkcijos randa duomenų failus.

1.4.4 Variantas C — Visual Studio

CMake projektą galima atidaryti tiesiogiai: **File** → **Open** → **CMake** → pasirinkti `CMakeLists.txt`.
Rankiniam projektui — **Properties** → **Configuration: Release, Platform: x64**:

Nustatymas	Reikšmė
C/C++ → Language → C++ Language Standard	ISO C++17 (/std:c++17)
C/C++ → Optimization → Optimization	/O2
C/C++ → Optimization → Whole Program Optimization	/GL
Linker → Optimization → Link Time Code Generation	/LTCG
Debugging → Working Directory	\$(ProjectDir)

Pastaba dėl /O3: MSVC tokio flag'o **neturi** — jo skalė yra /Od → /O1 → /O2, kur /O2 yra maksimalus lygis. GCC/Clang atitikmuo — -O2.

1.4.5 Unit testų paruošimas

```
mkdir -p tests
curl -L -o tests/catch.hpp \
  https://github.com/catchorg/Catch2/releases/download/v2.13.10/catch.hpp
```

Be `tests/catch.hpp` CMake parodys įspėjimą ir praleis testų taikinį — pagrindinė programa vis tiek sukompiliuos.

1.5 4. Klasių architektūra

```
zmogus (ABSTRAKTI)
protected:
    vardas_ , pavarde_
    konstruktoriai
public:
    virtual ~zmogus()
    vardas() / pavarde()
    pilnasVardas()
    setVardas() / setPavarde()

    = 0 galutinis()          ← grynai virtualūs
    = 0 print()
    = 0 read()
    = 0 tipas()
        public paveldėjimas

studentas (KONKRETI)
private:
    nd_ , egzaminas_
    galutinisVid_
    galutinisMed_
public:
    Rule of Five (5 metodai)
    override × 4
    parseFromLine()
    appendListTo()
    ==, !=, <, >, [], bool
```

1 pav. Klasių hierarchija (Visual Studio Class Designer)

1.5.1 Abstrakti klasė zmogus

Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais**:

1. **Keturi grynai virtualūs metodai** — `galutinis()`, `print()`, `read()`, `tipas()`
2. **protected konstruktoriai** — prieinami tik išvestinėms klasėms

Objekto sukurti neįmanoma:

```
zmogus z; // error C2259: cannot instantiate abstract class
zmogus* p = new zmogus(); // ta pati klaida
```

2 pav. Kompiliatoriaus klaida bandant sukurti *zmogus* objektą

1.5.2 Rule of Five su paveldėjimu

Kiekviena iš penkių funkcijų **privalo** iškvieisti atitinkamą bazinės klasės funkciją:

Metodas	Bazinės klasės kvietimas	Kas nutiktų be jo
Kopijavimo konstruktorius	<code>: zmogus(other)</code>	Vardas ir pavardė liktų tušti
Kopijavimo priskyrimas	<code>zmogus::operator=(other);</code>	Bazinė dalis nepasikeistų
Move konstruktorius	<code>: zmogus(std::move(other))</code>	Bazinė dalis būtų kopijuojama
Move priskyrimas	<code>zmogus::operator=(std::move(other));</code>	Ta pati problema
Destruktorius	Kviečiamas automatiškai	—

Trys subtilumai, kurių kompiliatorius nepagaus:

`std::move(other)` move konstruktoriuje. *other* yra pavadintas kintamasis, todėl pats savaime *lvalue* — net jei jo tipas *studentas&&*. Be `std::move` būtų iškvieistas *zmogus* **kopijavimo** konstruktorius.

Kvalifikuotas `zmogus::operator=`. Be `zmogus::` prefikso kompiliatorius parinktų `studentas::operator=` — įvyktų begalinė rekursija.

`noexcept` move operacijose. `std::vector` perskirstymo metu naudoja `std::move_if_noexcept` — be `noexcept` jis renka kopijavimo konstruktorių, ir sparta krenta kelis kartus.

1.5.3 Polimorfizmas

`operator<< ir operator>>` priima *zmogus&* ir kviečia virtualius `print()` / `read()`:

```
std::vector<std::unique_ptr<zmogus> zmones;
zmones.push_back(std::make_unique<studentas>("Ona", "Onaite", {10, 10}, 10));
```

```
for (const auto& z : zmones)
    std::cout << z->tipas() << ": " << *z << "\n";
```

Pridėjus naują išvestinę klasę ateityje, tie patys operatoriai veiks be pakeitimų.

3 pav. Polimorfinis konteineris ir virtualūs metodai

1.6 5. Dokumentacija

Dokumentacija generuojama **Doxygen** įrankiu iš kodo komentarų.

1.6.1 Generavimas

doxygen Doxyfile

Arba per CMake:

```
cmake --build build --target docs
```

1.6.2 Rezultatas

Formatas	Vieta	Kaip atidaryti
HTML	<code>docs/doxygen/↵ html/</code>	Naršyklėje atidaryti <code>index.html</code>
LaTeX	<code>docs/doxygen/↵ latex/</code>	Šaltiniai PDF generavimui
PDF	<code>docs/doxygen/refman.pdf</code>	Bet koku PDF skaitytuvu

4 pav. Sugeneruota HTML dokumentacija su paveldėjimo diagrama

1.6.3 PDF generavimas

Jei įdiegtas TeX Live arba MiKTeX:

```
cd docs/doxygen/latex
make.bat # Windows
```

1.6.4 Kas dokumentuota

Elementas	Aprašyta
Klasės <code>zmogus</code> , <code>studentas</code>	Paskirtis, paveldėjimo ryšys, naudojimo pavyzdžiai
Visi konstruktoriai	Parametrai, metamos išimtys
Rule of Five metodai	Kodėl reikia bazinės klasės kvietimo, kodėl <code>noexcept</code>
Įvesties/išvesties metodai	Formatai, skirtumas tarp greito ir universalaus varianto
Perdengti operatoriai	Kam gali prireikti būsimeis naudotojams
Šablonai (<code>calculate.h</code> , <code>file.h</code>)	Kaip veikia su skirtingais konteneriais

Naudojamos Doxygen žymos: `@file`, `@class`, `@brief`, `@param`, `@return`, `@throws`, `@note`, `@warning`, `@see`, `@code`/`@endcode`, `@par`, taip pat grupavimas su `@name` / `@{` / `@}`.

1.7 6. Unit testai

1.7.1 Kam jie reikalingi

Programoje jau yra rankinis testas (menu punktas 9), bet jis reikalauja paleisti programą ir žiūrėti į konsolę. Unit testai sprendžia tris problemas:

Problema	Sprendimas
Regresijos — pataisius vieną vietą sugenda kita	Testai paleidžiami po kiekvieno pakeitimo ir iškart parodo, kas sulūžo
Tylios klaidos — kodas kompiliuojasi, bet veikia neteisingai	Rule of Five klaidos būtent tokios: pamiršus <code>zmogus(other)</code> , vardai bus tušti, bet programa veiks
Rankinis tikrinimas — reikia žmogaus	Testai grąžina exit kodą, todėl tinka automatiniam paleidimui (CI)

Test Driven Development (TDD) — metodika, kai testai rašomi **prieš** realizaciją. Ciklas: raudona (testas krenta) → žalia (realizacija) → refactor. Šiame darbe testai rašyti po realizacijos, bet round-trip testas (`appendList` → `To` → `parseFromLine`) yra pavyzdys, kaip testas apibrėžia kontraktą tarp dviejų metodų — jis garantuoja, kad įrašytas failas bus teisingai nuskaitytas.

1.7.2 Pasirinktas framework — Catch2

Kodėl	
Vienas header failas	Nereikia atskiro build'o ar diegimo
<code>REQUIRE</code> vietoj <code>ASSERT_EQ</code>	Skaitomesnė sintaksė, veikia su bet kokia išraiška
<code>SECTION</code> blokai	Kiekvienas paleidžiamas su šviežiu <code>TEST_CASE</code> kontekstu

Kodėl	
Žymės ([rule5])	Galima paleisti tik dalį testų
Aiškūs pranešimai	Rodo tikslią eilutę ir abiejų pusių reikšmes

1.7.3 Paleidimas

```
./unitTests          # visi testai
./unitTests -s       # su detaliu išvedimu
./unitTests "[rule5]" # tik Rule of Five
./unitTests "[abstrakti]" # tik abstrakčios klasės
./unitTests -l       # testų sąrašas
ctest                # per CMake
```

1.7.4 Testų apimtis

Žymė	Ką tikrina	TEST_CASE	Sekcijų
[abstrakti]	Abstrakti klasė, paveldėjimas, virtualus destruktorius	1	4
↔ [konstruktoriai]↔	Visi trys konstruktoriai, validacija, ribinės reikšmės	2	7
[rule5]	Visi penki metodai + noexcept + destruktorius	5	20
↔ [polimorfizmas]↔	Virtualūs metodai, dynamic_cast, polimorfinis konteineris	1	4
[io]	>>, <<, parseFromLine, appendListTo, round-trip	3	8
[operatoriai]	==, !=, <, >, []	2	6
[skaiciavimas]	Vidurkis, mediana, kraštiniai atvejai	1	5
[seteriai]	Set'eriai, validacija, išvalyk()	1	5
[konteineriai]	vector perskirstymas, std::move, list/deque	1	3
Iš viso		17	62

5 pav. Catch2 unit testų išvestis

1.7.5 Įdomiausi testai

Deep copy patikra — ar objektai tikrai nepriklausomi:

```
studentas kopija(originalas);
kopija.setVardas("Pakeistas");
REQUIRE(originalas.vardas() == "Jonas");
```

Virtualaus destruktoriaus patikra — naudojami du objektų skaitikliai:

```
{
    std::unique_ptr<zmogus> p = std::make_unique<studentas>(...);
    REQUIRE(studentas::gyvuStudentu == priesStudentu + 1);
}
REQUIRE(studentas::gyvuStudentu == priesStudentu); // ~studentas() iškviestas
REQUIRE(zmogus::gyvuZmoniu == priesZmoniu);        // ~zmogus() iškviestas
```

noexcept patikra — statinė, kompiliavimo metu:

```
REQUIRE(std::is_nothrow_move_constructible<studentas>::value);
REQUIRE(std::is_nothrow_move_assignable<studentas>::value);
```

Round-trip patikra — svarbiausias I/O testas:

```
studentas originalas("Rasa", "Rasaite", {6, 7, 8}, 9);
std::string buferis;
originalas.appendListTo(buferis);

studentas nuskaitytas;
```

```
nuskaitytas.parseFromLine(buferis, 3, 1);
REQUIRE(nuskaitytas == originalas);
```

1.8 7. Spartos tyrimai

1.8.1 Techninė aplinka

Parametras	Reikšmė
Procesorius	Intel Core i7-8565U (4 branduoliai / 8 gijų, iki 4.6 GHz)
RAM	8 GB DDR4
Saugykla	250 GB SSD
OS	Windows 11
Kompiliatorius	MSVC (Visual Studio 2022), x64 Release, /O2

1.8.2 Konteinerių palyginimas

Matuoti trys žingsniai su tais pačiais failais. Kiekvienas dydis testuotas 3 kartus, pateikiami vidurkiai.

Rikiavimas (s)

Įrašų sk.	vector	list	deque
1 000	0.0021201	0.0008624	0.0024529
10 000	0.0336593	0.0061455	0.0412758
100 000	0.349848	0.0646853	0.378782
1 000 000	9.50295	1.43094	6.5229
10 000 000	73.956	16.4002	79.0366

`std::list` rikiavimas su 10 mln. studentų $\sim 4.5\times$ greitesnis nei `vector`. Priežastis: `list::sort()` naudoja merge sort ir tik perrikiuoja rodykles, o `std::sort` su `vector` fiziškai perkeldinėja objektus, kurių kiekvienas turi `std::string` ir `std::vector` viduje.

Skirstymas į grupes (s)

Įrašų sk.	vector	list	deque
100 000	0.0666329	0.0635167	0.0474443
1 000 000	1.54565	0.784462	0.529881
10 000 000	22.6479	12.5161	9.44885

1.8.3 Skirstymo strategijų palyginimas

Strategija	Metodas	Sudėtingumas
1	<code>partition_copy</code> į du naujus konteinerius	$O(n)$, bet $3\times$ atmintis
2	<code>stable_partition</code> + <code>erase</code> po vieną	$O(n^2)$ su <code>vector/deque</code>
3	<code>stable_partition</code> + blokinis <code>erase</code>	$O(n)$, $\sim 1.5\times$ atmintis

2 strategija su `vector` ir 100 000 studentų užtruko 215 sekundžių — $3\,234\times$ lėčiau nei 3 strategija (0.067 s). Priežastis: kiekvienas `erase` iš vidurio perstumia visus likusius elementus.

`std::list` šios problemos neturi — trynimasis $O(1)$, nes tai dvikryptė rodyklių sąsaja.

1.8.4 struct vs class ir optimizavimo flag'ai

Versija	Flag	.exe	100k (s)	1M (s)
class (MSVC)	/O1	120 KB	0.3979	4.7711
class (MSVC)	/O2	160 KB	0.4071	4.7157
struct (MinGW)	-O1	371 KB	0.5637	6.5547
struct (MinGW)	-O2	336 KB	0.7768	10.6278

Netikėtas rezultatas: **struct versija su -O2 yra 1.62x lėtesnė nei su -O1**. Visas skirtumas ateina iš failo rašymo. Tikėtina priežastis — agresyvus -O2 inline'inimas išpučia `std::setw/operator<< grandinę`, kuri cikle kviečiama 16 kartų vienai eilutei, ir kodas nebetelpa į instrukcijų cache.

Tai **hipotezė**, patikrinta tik netiesiogiai. Galutinis patvirtinimas reikalautų cache miss matavimo profiliu.

1.8.5 I/O optimizacija

Problema	Sprendimas	Efektas
<code>std::istreamstream</code> kiekvienai eilutei	<code>parseFromLine</code> dirba su <code>const std::string&</code>	10 mln. objektų konstravimų → 0
<code>operator>></code> skaičiams	<code>std::from_chars (C++17)</code>	Be locale, sentry, būsenos
Klaidos pranešimai kiekvienam pažymiui	Formuojami tik metant išimtį	~160 mln. konkatenuacijų → 0
<code>std::setw</code> manipulatoriai rašant	<code>std::to_chars + std::↔ string buferis</code>	
Kiekviena eilutė rašoma atskirai	Buferis, <code>file.write()</code> kas ~1 MB	

Rezultatas: failo nuskaitymas su 10 mln. studentų **nuo ~90 s iki ~30 s**.

1.9 8. Failų struktūra

```

.
LICENSE
README.md
CMakeLists.txt
Doxyfile
.gitignore
docs/
  *.png
  doxygen/
    html/
    latex/
    refman.pdf
tests/
  catch.hpp
  unitTests.cpp
OOP_2/
  data/
    studentail000.txt
    studentail0000.txt
    studentail00000.txt
  zmogus.h / zmogus.cpp
  studentas.h / studentas.cpp
  testStudentas.h / testStudentas.cpp
  main.h
  OOP_2.cpp
  menu.h / menu.cpp
  enter.h / enter.cpp
  generate.h / generate.cpp
  file.h / file.cpp
  calculate.h

# Programa + testai + docs taikinsys
# Doxygen konfigūracija
# IDE ir build failų filtras
# Ekranų kopijos ir dokumentacija
# Iliustracijos README failui
# Sugeneruota dokumentacija
# HTML formatai
# LaTeX šaltiniai
# Sukompiliuotas PDF
# Unit testai
# Catch2 framework (atsisiunčiamas)
# Testai
# Programos šaltinio failai
# Testiniai duomenų failai

# ABSTRAKTI bazinė klasė
# Išvestinė klasė : public zmogus
# Rankinis klasių testas (menu 9)
# splitResult<T>, deklaracijos
# main() + runProgram<Container>()
# Menu funkcijos
# Rankinio įvedimo funkcijos
# Generavimo funkcijos
# Failo I/O šablonai
# Skaičiavimo, rikiavimo šablonai

```

```
output.h           # outputStudentai<T>
print.h / print.cpp # Spausdinimo funkcijos
test.h / test.cpp   # Spartos testavimo funkcijos
```

Pastaba dėl data/. Testavimo funkcijos atidaro failus santykiniu keliu `data/studentaiN.txt`, todėl `data/` turi būti darbo aplanke. CMake tai sutvarko automatiškai (`POST_BUILD` kopijavimas). Dideli failai (1 mln. ir 10 mln. įrašų) į repozitoriją neįtraukti — juos galima sugeneruoti meniu punktu 5.

1.9.1 Repozitorijos švarumas

`.gitignore` filtruoja:

- Visual Studio projekto failus (`.vcxproj`, `.sln`, `.cd`, `.vs/`)
- Build aplankus (`build/`, `x64/`, `Debug/`, `Release/`)
- Kompiliavimo rezultatus (`.exe`, `.obj`, `.pdb`)
- Programos generuojamus failus (`teststudentai*.txt`)

Repozitorijoje lieka **tik šaltinio failai**, dokumentacija ir konfigūracija.

1.10 Licencija

Žr. `[LICENSE](LICENSE)`.

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

tests	??
benchmark.cpp	??
unitTests.cpp	??
Vectortests.cpp	??

skyrius 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

tests/ benchmark.cpp	
Spartos analizė: std::vector vs nuosava Vector klasė	??
tests/ unitTests.cpp	
Unit testai klasėms žmogus ir studentas (Catch2 framework)	??
tests/ Vectortests.cpp	
Unit testai nuosavai Vector klasei (Catch2)	??

skyrius 4

Direktorių dokumentacija

4.1 tests Direktorijos aprašas

Failai

- failas [benchmark.cpp](#)
Spartos analizė: std::vector vs nuosava Vector klasė.
- failas [unitTests.cpp](#)
Unit testai klasėms žmogus ir studentas (Catch2 framework).
- failas [Vectortests.cpp](#)
Unit testai nuosavai Vector klasei (Catch2).

skyrius 5

Failo Dokumentacija

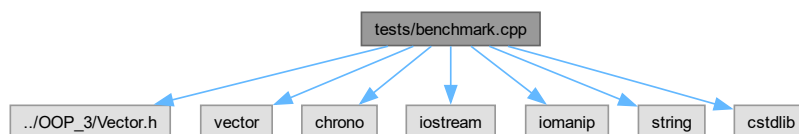
5.1 README.md Failo Nuoroda

5.2 tests/benchmark.cpp Failo Nuoroda

Spartos analizė: std::vector vs nuosava Vector klasė.

```
#include "../OOP_3/Vector.h"
#include <vector>
#include <chrono>
#include <iostream>
#include <iomanip>
#include <string>
#include <cstdlib>
```

Įtraukimo priklausomybių diagrama benchmark.cpp:



Tipų apibrėžimai

- using [Laikrodis](#) = std::chrono::high_resolution_clock
- using [Sekundes](#) = std::chrono::duration<double>

Funkcijos

- double [matuokStdVector](#) (unsigned int sz)
Matuoja std::vector užpildymo laiką.
- double [matuokVector](#) (unsigned int sz)
Matuoja nuosavos Vector užpildymo laiką.
- void [tyrimas1](#) ()
- unsigned long long [skaiciuokStdVector](#) (unsigned int sz)
Skaičiuoja std::vector perskirstymus.
- unsigned long long [skaiciuokVector](#) (unsigned int sz)
Skaičiuoja nuosavos Vector perskirstymus.
- void [tyrimas2](#) ()
- void [tyrimas3](#) ()

- int `main` (int argc, char *argv[])

5.2.1 Smulkus aprašymas

Spartos analizė: `std::vector` vs nuosava `Vector` klasė.

Atlieka du tyrimus:

1. `push_back()` spartos palyginimas su 10 ... 10 elementų
2. Atminties perskirstymų skaičiavimas

Kompiliavimas: `g++ -std=c++17 -O2 benchmark.cpp -o benchmark`

Paleidimas: `./benchmark` — visi tyrimai `./benchmark 1` — tik `push_back` tyrimas `./benchmark 2` — tik perskirstymų tyrimas

Ispėjimas

100 000 000 elementų testas naudoja ~400 MB kiekvienam konteneriui. Su 8 GB RAM tai priimtina, bet mažesnėse sistemose gali prireikti praleisti paskutinį dydį.

Apibrėžimas faile `benchmark.cpp`.

5.2.2 Tipų apibrėžimų Dokumentacija

5.2.2.1 Laikrodis

using `Laikrodis` = `std::chrono::high_resolution_clock`

Apibrėžimas failo `benchmark.cpp` eilutėje 31.

5.2.2.2 Sekundes

using `Sekundes` = `std::chrono::duration<double>`

Apibrėžimas failo `benchmark.cpp` eilutėje 32.

5.2.3 Funkcijos Dokumentacija

5.2.3.1 `matuokStdVector()`

```
double matuokStdVector (
    unsigned int sz)
```

Matuoja `std::vector` užpildymo laiką.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Trukmė sekundėmis

Apibrėžimas failo `benchmark.cpp` eilutėje 43.

Naudojamas `tyrimas1()`.

5.2.3.2 `matuokVector()`

```
double matuokVector (
    unsigned int sz)
```

Matuoja nuosavos `Vector` užpildymo laiką.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Trukmė sekundėmis

Apibrėžimas failo [benchmark.cpp](#) eilutėje 60.

Naudojamas [tyrimas1\(\)](#).

5.2.3.3 tyrimas1()

```
void tyrimas1 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 72.

Ryšiai [matuokStdVector\(\)](#), ir [matuokVector\(\)](#).

Naudojamas [main\(\)](#).

5.2.3.4 skaiciuokStdVector()

```
unsigned long long skaiciuokStdVector (  
    unsigned int sz)
```

Skaičiuoja std::vector perskirstymus.

Perskirstymas aptinkamas stebint capacity() pokytį — jis įvyksta tada, kai capacity() == size() ir pridedamas naujas elementas.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Perskirstymų skaičius

Apibrėžimas failo [benchmark.cpp](#) eilutėje 117.

Naudojamas [tyrimas2\(\)](#).

5.2.3.5 skaiciuokVector()

```
unsigned long long skaiciuokVector (  
    unsigned int sz)
```

Skaičiuoja nuosavos Vector perskirstymus.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Perskirstymų skaičius

Apibrėžimas failo [benchmark.cpp](#) eilutėje 138.

Naudojamas [tyrimas2\(\)](#).

5.2.3.6 tyrimas2()

```
void tyrimas2 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 154.

Ryšiai [skaiciuokStdVector\(\)](#), ir [skaiciuokVector\(\)](#).

Naudojamas [main\(\)](#).

5.2.3.7 tyrimas3()

```
void tyrimas3 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 187.

Naudojamas [main\(\)](#).

5.2.3.8 main()

```
int main (
    int argc,
    char * argv[])
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 217.

Ryšiai [tyrimas1\(\)](#), [tyrimas2\(\)](#), ir [tyrimas3\(\)](#).

5.3 benchmark.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file benchmark.cpp
00003  * @brief Spartos analizė: std::vector vs nuosava Vector klasė.
00004  *
00005  * Atlieka du tyrimus:
00006  * 1. push_back() spartos palyginimas su 10 ... 10 elementų
00007  * 2. Atminties perskirstymų skaičiavimas
00008  *
00009  * Kompiliavimas:
00010  * g++ -std=c++17 -O2 benchmark.cpp -o benchmark
00011  *
00012  * Paleidimas:
00013  * ./benchmark -- visi tyrimai
00014  * ./benchmark 1 -- tik push_back tyrimas
00015  * ./benchmark 2 -- tik perskirstymų tyrimas
00016  *
00017  * @warning 100 000 000 elementų testas naudoja ~400 MB kiekvienam
00018  * konteineriui. Su 8 GB RAM tai priimtina, bet mažesnėse
00019  * sistemose gali prireikti praleisti paskutinį dydį.
00020  */
00021
00022 #include "../OOP_3/Vector.h"
00023
00024 #include <vector>
00025 #include <chrono>
00026 #include <iostream>
00027 #include <iomanip>
00028 #include <string>
00029 #include <cstdlib>
00030
00031 using Laikrodis = std::chrono::high_resolution_clock;
00032 using Sekundes = std::chrono::duration<double>;
00033
00034 // =====
00035 // 1 TYRIMAS -- push_back sparta
00036 // =====
00037
00038 /**
00039  * @brief Matuoja std::vector užpildymo laiką.
00040  * @param sz Elementų skaičius
00041  * @return Trukmė sekundėmis
00042  */
00043 double matuokStdVector(unsigned int sz)
00044 {
00045     auto pradzia = Laikrodis::now();
00046
00047     std::vector<int> v1;
00048     for (unsigned int i = 1; i <= sz; ++i)
00049         v1.push_back(static_cast<int>(i));
00050
00051     auto pabaiga = Laikrodis::now();
00052     return Sekundes(pabaiga - pradzia).count();
00053 }
00054
00055 /**
00056  * @brief Matuoja nuosavos Vector užpildymo laiką.
00057  * @param sz Elementų skaičius
00058  * @return Trukmė sekundėmis
00059  */
00060 double matuokVector(unsigned int sz)
00061 {
```

```

00062     auto pradzia = Laikrodis::now();
00063
00064     Vector<int> v2;
00065     for (unsigned int i = 1; i <= sz; ++i)
00066         v2.push_back(static_cast<int>(i));
00067
00068     auto pabaiga = Laikrodis::now();
00069     return Sekundes(pabaiga - pradzia).count();
00070 }
00071
00072 void tyrimas1()
00073 {
00074     const unsigned int dydziai[] = {
00075         10000u, 100000u, 1000000u, 10000000u, 100000000u
00076     };
00077
00078     std::cout << "\n=====\\n";
00079     std::cout << " 1 TYRIMAS - push_back() sparta\\n";
00080     std::cout << "=====\\n\\n";
00081
00082     std::cout << std::left << std::setw(14) << "Elementu"
00083         << std::right << std::setw(14) << "std::vector"
00084         << std::setw(14) << "Vector"
00085         << std::setw(12) << "Santykis" << "\\n";
00086     std::cout << std::string(54, '-') << "\\n";
00087
00088     for (unsigned int sz : dydziai) {
00089         const double t1 = matuokStdVector(sz);
00090         const double t2 = matuokVector(sz);
00091         const double santykis = t1 > 0.0 ? t2 / t1 : 0.0;
00092
00093         std::cout << std::left << std::setw(14) << sz
00094             << std::right << std::fixed << std::setprecision(6)
00095             << std::setw(14) << t1
00096             << std::setw(14) << t2
00097             << std::setprecision(2) << std::setw(11) << santykis << "x"
00098             << "\\n";
00099     }
00100
00101     std::cout << "\\nSantykis > 1.0 reiskia, kad Vector letesnis uz std::vector.\\n\\n";
00102 }
00103
00104 // =====
00105 // 2 TYRIMAS -- atminties perskirstymai
00106 // =====
00107
00108 /**
00109  * @brief Skaičiuoja std::vector perskirstymus.
00110  *
00111  * Perskirstymas aptinkamas stebint capacity() pokytį -- jis įvyksta
00112  * tada, kai capacity() == size() ir pridamas naujas elementas.
00113  *
00114  * @param sz Elementų skaičius
00115  * @return Perskirstymų skaičius
00116  */
00117 unsigned long long skaiciukStdVector(unsigned int sz)
00118 {
00119     std::vector<int> v;
00120     unsigned long long perskirstymu = 0;
00121     std::size_t senaTalpa = v.capacity();
00122
00123     for (unsigned int i = 1; i <= sz; ++i) {
00124         v.push_back(static_cast<int>(i));
00125         if (v.capacity() != senaTalpa) {
00126             ++perskirstymu;
00127             senaTalpa = v.capacity();
00128         }
00129     }
00130     return perskirstymu;
00131 }
00132
00133 /**
00134  * @brief Skaičiuoja nuosavos Vector perskirstymus.
00135  * @param sz Elementų skaičius
00136  * @return Perskirstymų skaičius
00137  */
00138 unsigned long long skaiciukVector(unsigned int sz)
00139 {
00140     Vector<int> v;
00141     unsigned long long perskirstymu = 0;
00142     std::size_t senaTalpa = v.capacity();
00143
00144     for (unsigned int i = 1; i <= sz; ++i) {
00145         v.push_back(static_cast<int>(i));
00146         if (v.capacity() != senaTalpa) {
00147             ++perskirstymu;
00148             senaTalpa = v.capacity();

```

```

00149     }
00150     }
00151     return perskirstymu;
00152 }
00153
00154 void tyrimas2()
00155 {
00156     const unsigned int dydziai[] = {
00157         10000u, 100000u, 1000000u, 10000000u, 100000000u
00158     };
00159
00160     std::cout << "\n=====\\n";
00161     std::cout << " 2 TYRIMAS - atminties perskirstymu skaicius\\n";
00162     std::cout << "=====\\n\\n";
00163
00164     std::cout << std::left << std::setw(14) << "Elementu"
00165         << std::right << std::setw(14) << "std::vector"
00166         << std::setw(14) << "Vector" << "\\n";
00167     std::cout << std::string(42, '-') << "\\n";
00168
00169     for (unsigned int sz : dydziai) {
00170         const auto p1 = skaiciuokStdVector(sz);
00171         const auto p2 = skaiciuokVector(sz);
00172
00173         std::cout << std::left << std::setw(14) << sz
00174             << std::right << std::setw(14) << p1
00175             << std::setw(14) << p2 << "\\n";
00176     }
00177
00178     std::cout << "\\nPerskirstymas ivyksta, kai capacity() == size() ir\\n"
00179         << "pridedamas naujas elementas. Su dvigubinimo strategija\\n"
00180         << "perskirstymu skaicius artimas log2(n).\\n\\n";
00181 }
00182
00183 // =====
00184 // Papildomas tyrimas -- talpos augimo seka
00185 // =====
00186
00187 void tyrimas3()
00188 {
00189     std::cout << "\\n=====\\n";
00190     std::cout << " 3 TYRIMAS - talpos augimo seka (pirmi 20 elementu)\\n";
00191     std::cout << "=====\\n\\n";
00192
00193     std::cout << std::left << std::setw(10) << "size()"
00194         << std::setw(20) << "std::vector cap"
00195         << std::setw(20) << "Vector cap" << "\\n";
00196     std::cout << std::string(50, '-') << "\\n";
00197
00198     std::vector<int> sv;
00199     Vector<int> v;
00200
00201     for (int i = 1; i <= 20; ++i) {
00202         sv.push_back(i);
00203         v.push_back(i);
00204         std::cout << std::left << std::setw(10) << i
00205             << std::setw(20) << sv.capacity()
00206             << std::setw(20) << v.capacity() << "\\n";
00207     }
00208
00209     std::cout << "\\nMSVC naudoja 1.5x augimo koeficienta, GCC/Clang - 2x.\\n"
00210         << "Nuosava Vector naudoja 2x.\\n\\n";
00211 }
00212
00213 // =====
00214 // main
00215 // =====
00216
00217 int main(int argc, char* argv[])
00218 {
00219     std::cout << "\\n";
00220     std::cout << "#####\\n";
00221     std::cout << "# std::vector vs Vector - spartos analize #\\n";
00222     std::cout << "#####\\n";
00223
00224     if (argc > 1) {
00225         const int nr = std::atoi(argv[1]);
00226         switch (nr) {
00227             case 1: tyrimas1(); break;
00228             case 2: tyrimas2(); break;
00229             case 3: tyrimas3(); break;
00230             default:
00231                 std::cout << "\\nNezinomas tyrimo numeris. Galimi: 1, 2, 3\\n";
00232                 return 1;
00233         }
00234     }
00235     else {

```



```

00236         tyrimas3();
00237         tyrimas1();
00238         tyrimas2();
00239     }
00240
00241     return 0;
00242 }

```

5.4 tests/unitTests.cpp Failo Nuoroda

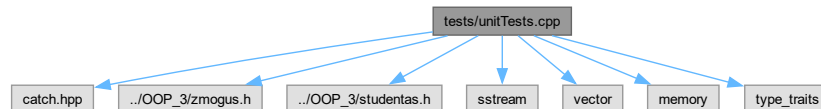
Unit testai klasėms žmogus ir studentas (Catch2 framework).

```

#include "catch.hpp"
#include "../OOP_3/zmogus.h"
#include "../OOP_3/studentas.h"
#include <sstream>
#include <vector>
#include <memory>
#include <type_traits>

```

Įtraukimo priklausomybių diagrama unitTests.cpp:



Apibrėžimai

- #define CATCH_CONFIG_MAIN

Funkcijos

- TEST_CASE ("zmogus yra abstrakti klase", "[abstrakti]")
- TEST_CASE ("Konstruktoriai", "[konstruktoriai]")
- TEST_CASE ("Konstruktoriaus validacija", "[konstruktoriai][validacija]")
- TEST_CASE ("Rule of Five: kopijavimo konstruktorius", "[rule5][kopijavimas]")
- TEST_CASE ("Rule of Five: kopijavimo priskyrimo operatorius", "[rule5][kopijavimas]")
- TEST_CASE ("Rule of Five: perkeliimo konstruktorius", "[rule5][perkelimas]")
- TEST_CASE ("Rule of Five: perkeliimo priskyrimo operatorius", "[rule5][perkelimas]")
- TEST_CASE ("Rule of Five: noexcept specifikatorius", "[rule5][noexcept]")
- TEST_CASE ("Rule of Five: destruktorius", "[rule5][destruktorius]")
- TEST_CASE ("Polimorfizmas", "[polimorfizmas]")
- TEST_CASE ("Ivesties operatorius >>", "[io]")
- TEST_CASE ("Išvesties operatorius <<", "[io]")
- TEST_CASE ("Failo I/O: parseFromLine ir appendListTo", "[io][failai]")
- TEST_CASE ("Lyginimo operatoriai", "[operatoriai]")
- TEST_CASE ("Indeksavimo operatorius []", "[operatoriai]")
- TEST_CASE ("calculateGalutinis", "[skaiciavimas]")
- TEST_CASE ("Seteriai", "[seteriai]")
- TEST_CASE ("Veikimas su STL konteineriais", "[konteineriai]")

5.4.1 Smulkus aprašymas

Unit testai klasėms žmogus ir studentas (Catch2 framework).

Testai suskirstyti į sekcijas pagal testuojamą klasės aspektą. Pagrindinis dėmesys — penkių metodų taisyklė (Rule of Five), nes būtent ji lengviausiai sugenda tyliai: pamiršus bazinės klasės kvietimą arba noexcept, programa vis tiek kompiliuos ir veiks, tik lėčiau arba su prarastais duomenimis.

Paleidimas: `./unitTests` — visi testai `./unitTests -s` — su detaliu išvedimu `./unitTests "[rule5]"` — tik Rule of Five testai `./unitTests -l` — testų sąrašas

Apibrėžimas faile [unitTests.cpp](#).

5.4.2 Apibrėžimų Dokumentacija

5.4.2.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 18.

5.4.3 Funkcijos Dokumentacija

5.4.3.1 TEST_CASE() [1/18]

```
TEST_CASE (
    "zmogus yra abstrakti klase" ,
    "" [abstrakti])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 52.

5.4.3.2 TEST_CASE() [2/18]

```
TEST_CASE (
    "Konstruktoriai" ,
    "" [konstruktoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 82.

5.4.3.3 TEST_CASE() [3/18]

```
TEST_CASE (
    "Konstruktoriaus validacija" ,
    "" [konstruktoriai][validacija])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 114.

5.4.3.4 TEST_CASE() [4/18]

```
TEST_CASE (
    "Rule of Five: kopijavimo konstruktorius" ,
    "" [rule5][kopijavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 150.

5.4.3.5 TEST_CASE() [5/18]

```
TEST_CASE (
    "Rule of Five: kopijavimo priskyrimo operatorius" ,
    "" [rule5][kopijavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 181.

5.4.3.6 TEST_CASE() [6/18]

```
TEST_CASE (
    "Rule of Five: perkeliimo konstruktorius" ,
    "" [rule5][perkelimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 221.

5.4.3.7 TEST_CASE() [7/18]

```
TEST_CASE (
    "Rule of Five: perkeliimo priskyrimo operatorius" ,
    "" [rule5][perkelimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 259.

5.4.3.8 TEST_CASE() [8/18]

```
TEST_CASE (
    "Rule of Five: noexcept specifikatorius" ,
    "" [rule5][noexcept])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 285.

5.4.3.9 TEST_CASE() [9/18]

```
TEST_CASE (
    "Rule of Five: destruktorius" ,
    "" [rule5][destruktorius])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 303.

5.4.3.10 TEST_CASE() [10/18]

```
TEST_CASE (
    "Polimorfizmas" ,
    "" [polimorfizmas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 347.

5.4.3.11 TEST_CASE() [11/18]

```
TEST_CASE (
    "Ivesties operatorius >>" ,
    "" [io])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 395.

5.4.3.12 TEST_CASE() [12/18]

```
TEST_CASE (
    "Ivesties operatorius <<" ,
    "" [io])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 417.

5.4.3.13 TEST_CASE() [13/18]

```
TEST_CASE (
    "Failo I/O: parseFromLine ir appendListTo" ,
    "" [io][failai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 439.

5.4.3.14 TEST_CASE() [14/18]

```
TEST_CASE (
    "Lyginimo operatoriai" ,
    "" [operatoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 493.

5.4.3.15 TEST_CASE() [15/18]

```
TEST_CASE (
    "Indeksavimo operatorius " [],
    "" [operatoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 525.

5.4.3.16 TEST_CASE() [16/18]

```
TEST_CASE (
    "calculateGalutinis" ,
    "" [skaiciavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 545.

5.4.3.17 TEST_CASE() [17/18]

```
TEST_CASE (
    "Seteriai" ,
    "" [seteriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 593.

5.4.3.18 TEST_CASE() [18/18]

```
TEST_CASE (
    "Veikimas su STL konteineriais" ,
    "" [konteineriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 642.

5.5 unitTests.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file unitTests.cpp
00003  * @brief Unit testai klasėms zmogus ir studentas (Catch2 framework).
00004  *
00005  * Testai suskirstyti į sekcijas pagal testuojamą klasės aspektą.
00006  * Pagrindinis dėmesys -- penkių metodų taisyklė (Rule of Five),
00007  * nes būtent ji lengviausiai sugenda tyliai: pamiršus bazinės klasės
00008  * kvietimą arba noexcept, programa vis tiek kompiliuos ir veiks,
00009  * tik lėčiau arba su prarastais duomenimis.
00010  *
00011  * Paleidimas:
00012  *     ./unitTests           -- visi testai
00013  *     ./unitTests -s       -- su detaliu išvedimu
00014  *     ./unitTests "[rule5]" -- tik Rule of Five testai
00015  *     ./unitTests -l       -- testų sąrašas
00016  */
00017
00018 #define CATCH_CONFIG_MAIN
00019 #include "catch.hpp"
00020
00021 #include "../OOP_3/zmogus.h"
00022 #include "../OOP_3/studentas.h"
00023
00024 #include <sstream>
00025 #include <vector>
00026 #include <memory>
00027 #include <type_traits>
00028
00029 // =====
00030 // Pagalbinės funkcijos
00031 // =====
00032
00033 namespace {
00034
00035     /**
00036      * @brief Sukuria testinį studentą su žinomomis reikšmėmis.
00037      *
00038      * ND: 5,5,5,5,5 → vidurkis 5.0, mediana 5.0
00039      * Egzaminas 10 → galutinis = 0.4*5.0 + 0.6*10 = 8.0
00040      */
00041     studentas kurkTestinį()
```

```

00042     {
00043         return studentas("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00044     }
00045 } // anoniminis namespace
00046
00047 // =====
00048 // 1. ABSTRAKČIOJI KLASĖ
00049 // =====
00050
00051
00052 TEST_CASE("zmogus yra abstrakti klase", "[abstrakti]")
00053 {
00054     SECTION("is_abstract patikra") {
00055         REQUIRE(std::is_abstract<zmogus>::value);
00056         REQUIRE_FALSE(std::is_abstract<studentas>::value);
00057     }
00058
00059     SECTION("zmogus objekto sukurti neimanoma") {
00060         // Sis kodas NESIKOMPILIUOTU:
00061         //     zmogus z;
00062         //     zmogus* p = new zmogus();
00063         // Todel patikra atliekama statiskai:
00064         REQUIRE_FALSE(std::is_default_constructible<zmogus>::value);
00065         REQUIRE(std::is_default_constructible<studentas>::value);
00066     }
00067
00068     SECTION("paveldejimo rysys") {
00069         REQUIRE(std::is_base_of<zmogus, studentas>::value);
00070         REQUIRE(std::is_convertible<studentas*, zmogus*>::value);
00071     }
00072
00073     SECTION("virtualus destruktorius") {
00074         REQUIRE(std::has_virtual_destructor<zmogus>::value);
00075     }
00076 }
00077
00078 // =====
00079 // 2. KONSTRUKTORIAI
00080 // =====
00081
00082 TEST_CASE("Konstruktoriai", "[konstruktoriai]")
00083 {
00084     SECTION("numatytasis konstruktorius inicijuoja visus laukus") {
00085         studentas s;
00086         REQUIRE(s.vardas().empty());
00087         REQUIRE(s.pavarde().empty());
00088         REQUIRE(s.nd().empty());
00089         REQUIRE(s.egzaminas() == 0);
00090         REQUIRE(s.galutinisVid() == Approx(0.0));
00091         REQUIRE(s.galutinisMed() == Approx(0.0));
00092         REQUIRE_FALSE(static_cast<bool>(s));
00093     }
00094
00095     SECTION("pilnas konstruktorius issaugo duomenis") {
00096         studentas s("Jonas", "Jonaitis", { 5, 6, 7 }, 8);
00097         REQUIRE(s.vardas() == "Jonas");
00098         REQUIRE(s.pavarde() == "Jonaitis");
00099         REQUIRE(s.nd().size() == 3);
00100         REQUIRE(s.egzaminas() == 8);
00101         REQUIRE(static_cast<bool>(s));
00102     }
00103
00104     SECTION("konstruktorius is srauto") {
00105         std::istringstream iss("Petras Petraitis 8 9 10 7");
00106         studentas s(iss);
00107         REQUIRE(s.vardas() == "Petras");
00108         REQUIRE(s.pavarde() == "Petraitis");
00109         REQUIRE(s.nd().size() == 3);
00110         REQUIRE(s.egzaminas() == 7);
00111     }
00112 }
00113
00114 TEST_CASE("Konstruktoriaus validacija", "[konstruktoriai][validacija]")
00115 {
00116     SECTION("bazine klase meta invalid_argument tusciam vardui") {
00117         REQUIRE_THROWS_AS(studentas("", "Jonaitis", { 5 }, 10),
00118             std::invalid_argument);
00119     }
00120
00121     SECTION("bazine klase meta invalid_argument tusciai pavardei") {
00122         REQUIRE_THROWS_AS(studentas("Jonas", "", { 5 }, 10),
00123             std::invalid_argument);
00124     }
00125
00126     SECTION("isvestine klase meta runtime_error blogam ND pazymiui") {
00127         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5, 99 }, 10),
00128             std::runtime_error);
00128     }

```

```

00129         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 0 }, 10),
00130                               std::runtime_error);
00131     }
00132
00133     SECTION("isvestine klase meta runtime_error blogam egzaminui") {
00134         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5 }, 0),
00135                             std::runtime_error);
00136         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5 }, 11),
00137                             std::runtime_error);
00138     }
00139
00140     SECTION("ribines reiksmes 1 ir 10 yra leidziamos") {
00141         REQUIRE_NOTHROW(studentas("Jonas", "Jonaitis", { 1, 10 }, 1));
00142         REQUIRE_NOTHROW(studentas("Jonas", "Jonaitis", { 1, 10 }, 10));
00143     }
00144 }
00145
00146 // =====
00147 // 3. RULE OF FIVE -- pagrindinis testų blokas
00148 // =====
00149
00150 TEST_CASE("Rule of Five: kopijavimo konstruktorius", "[rule5][kopijavimas]")
00151 {
00152     studentas originalas = kurkTestini();
00153     originalas.calculateGalutinis();
00154
00155     studentas kopija(originalas);
00156
00157     SECTION("bazines klases laukai nukopijuoti") {
00158         // Daznausia paveldejimo klaida: pamirsus zmogus(other) kvietima,
00159         // vardas ir pavarde liktu tusti
00160         REQUIRE(kopija.vardas() == "Jonas");
00161         REQUIRE(kopija.pavarde() == "Jonaitis");
00162     }
00163
00164     SECTION("isvestines klases laukai nukopijuoti") {
00165         REQUIRE(kopija.nd() == originalas.nd());
00166         REQUIRE(kopija.egzaminas() == originalas.egzaminas());
00167         REQUIRE(kopija.galutinisVid() == Approx(originalas.galutinisVid()));
00168         REQUIRE(kopija.galutinisMed() == Approx(originalas.galutinisMed()));
00169     }
00170
00171     SECTION("kopija yra GILI (deep copy) -- objektai nepriklausomi") {
00172         kopija.setVardas("Pakeistas");
00173         REQUIRE(originalas.vardas() == "Jonas");
00174
00175         kopija.addPazymys(1);
00176         REQUIRE(originalas.nd().size() == 5);
00177         REQUIRE(kopija.nd().size() == 6);
00178     }
00179 }
00180
00181 TEST_CASE("Rule of Five: kopijavimo priskyrimo operatorius", "[rule5][kopijavimas]")
00182 {
00183     studentas a = kurkTestini();
00184     a.calculateGalutinis();
00185
00186     SECTION("priskyrimas nukopijuoja visus laukus") {
00187         studentas b;
00188         b = a;
00189         REQUIRE(b.vardas() == a.vardas());
00190         REQUIRE(b.pavarde() == a.pavarde());
00191         REQUIRE(b.nd() == a.nd());
00192         REQUIRE(b.galutinisVid() == Approx(a.galutinisVid()));
00193     }
00194
00195     SECTION("priskyrimas sau nesugadina objekto") {
00196         // Be patikros this == &other objektas galetu buti sugadintas
00197         a = a;
00198         REQUIRE(a.vardas() == "Jonas");
00199         REQUIRE(a.nd().size() == 5);
00200         REQUIRE(a.egzaminas() == 10);
00201     }
00202
00203     SECTION("grandinis priskyrimas veikia") {
00204         // Reikalauja, kad operator= grazintu *this
00205         studentas b, c;
00206         c = b = a;
00207         REQUIRE(b.vardas() == "Jonas");
00208         REQUIRE(c.vardas() == "Jonas");
00209         REQUIRE(c.nd() == a.nd());
00210     }
00211
00212     SECTION("priskyrimas perrastos senus duomenis") {
00213         studentas b("Senas", "Senaitis", { 1, 1, 1 }, 1);
00214         b = a;
00215         REQUIRE(b.vardas() == "Jonas");

```

```

00216         REQUIRE(b.nd().size() == 5);
00217         REQUIRE(b.egzaminas() == 10);
00218     }
00219 }
00220
00221 TEST_CASE("Rule of Five: perkeliimo konstruktorius", "[rule5][perkelimas]")
00222 {
00223     studentas saltinis = kurkTestini();
00224     saltinis.calculateGalutinis();
00225     const double tiketinasVid = saltinis.galutinisVid();
00226
00227     studentas tikslas(std::move(saltinis));
00228
00229     SECTION("bazines klases laukai perkelti") {
00230         // Reikalauja zmogus(std::move(other)) -- be std::move butu
00231         // iskviestas KOPIJAVIMO, ne perkeliimo konstruktorius
00232         REQUIRE(tikslas.vardas() == "Jonas");
00233         REQUIRE(tikslas.pavarde() == "Jonaitis");
00234     }
00235
00236     SECTION("isvestines klases laukai perkelti") {
00237         REQUIRE(tikslas.nd().size() == 5);
00238         REQUIRE(tikslas.egzaminas() == 10);
00239         REQUIRE(tikslas.galutinisVid() == Approx(tiketinasVid));
00240     }
00241
00242     SECTION("saltinis lieka galiojancioje tuscioje busenoje") {
00243         REQUIRE(saltinis.vardas().empty());
00244         REQUIRE(saltinis.pavarde().empty());
00245         REQUIRE(saltinis.nd().empty());
00246         REQUIRE(saltinis.egzaminas() == 0);
00247         REQUIRE_FALSE(static_cast<bool>(saltinis));
00248     }
00249
00250     SECTION("istustinta saltini saugu naudoti is naujo") {
00251         saltinis.setVardas("Naujas");
00252         saltinis.setPavarde("Naujaitis");
00253         saltinis.setEgzaminas(5);
00254         REQUIRE(saltinis.vardas() == "Naujas");
00255         REQUIRE(static_cast<bool>(saltinis));
00256     }
00257 }
00258
00259 TEST_CASE("Rule of Five: perkeliimo priskyrimo operatorius", "[rule5][perkelimas]")
00260 {
00261     SECTION("perkelimas i esama objekta") {
00262         studentas saltinis = kurkTestini();
00263         saltinis.calculateGalutinis();
00264         studentas tikslas("Senas", "Senaitis", { 1, 1 }, 1);
00265
00266         tikslas = std::move(saltinis);
00267
00268         REQUIRE(tikslas.vardas() == "Jonas");
00269         REQUIRE(tikslas.nd().size() == 5);
00270         REQUIRE(tikslas.egzaminas() == 10);
00271         REQUIRE(saltinis.vardas().empty());
00272         REQUIRE(saltinis.nd().empty());
00273     }
00274
00275     SECTION("perkelimo priskyrimas sau nesukelia luzimo") {
00276         studentas x = kurkTestini();
00277         x = std::move(x);
00278         // Standartas negarantuoja konkreios reiksmes po self-move,
00279         // tik kad objektas liktu galiojancioje busenoje
00280         REQUIRE_NOTHROW(x.vardas());
00281         REQUIRE_NOTHROW(x.nd().size());
00282     }
00283 }
00284
00285 TEST_CASE("Rule of Five: noexcept specifikatorius", "[rule5][noexcept]")
00286 {
00287     // Be noexcept std::vector persikirstymo metu naudotu KOPIJAVIMO,
00288     // ne perkeliimo konstruktoriu (std::move_if_noexcept)
00289     SECTION("move konstruktorius yra noexcept") {
00290         REQUIRE(std::is_nothrow_move_constructible<studentas>::value);
00291     }
00292
00293     SECTION("move priskyrimas yra noexcept") {
00294         REQUIRE(std::is_nothrow_move_assignable<studentas>::value);
00295     }
00296
00297     SECTION("bazine klase taip pat noexcept") {
00298         REQUIRE(std::is_nothrow_move_constructible<zmogus>::value);
00299         REQUIRE(std::is_nothrow_move_assignable<zmogus>::value);
00300     }
00301 }
00302

```

```

00303 TEST_CASE("Rule of Five: destruktorius", "[rule5][destruktorius]")
00304 {
00305     const int priesStudentu = studentas::gyvuStudentu;
00306     const int priesZmoniu = zmogus::gyvuZmoniu;
00307
00308     SECTION("objektai sunaikinami isejus is srities") {
00309         {
00310             studentas a = kurkTestini();
00311             studentas b = kurkTestini();
00312             studentas c(a);
00313             REQUIRE(studentas::gyvuStudentu == priesStudentu + 3);
00314             REQUIRE(zmogus::gyvuZmoniu == priesZmoniu + 3);
00315         }
00316         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00317         REQUIRE(zmogus::gyvuZmoniu == priesZmoniu);
00318     }
00319
00320     SECTION("virtualus destruktorius naikinant per bazines klases rodykle") {
00321         {
00322             // Be virtual destruktoriaus ~studentas() NEBUTU iskviestas,
00323             // ir nd_ vektorius liktu neatlaisvintas
00324             std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00325                 "Testas", "Testaitis", Vector<int>{ 5, 5 }, 5);
00326             REQUIRE(studentas::gyvuStudentu == priesStudentu + 1);
00327         }
00328         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00329         REQUIRE(zmogus::gyvuZmoniu == priesZmoniu);
00330     }
00331
00332     SECTION("nera objektu nutekejimo naudojant konteinerius") {
00333         {
00334             std::vector<studentas> v;
00335             for (int i = 0; i < 10; ++i)
00336                 v.push_back(kurkTestini());
00337             REQUIRE(studentas::gyvuStudentu == priesStudentu + 10);
00338         }
00339         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00340     }
00341 }
00342
00343 // =====
00344 // 4. POLIMORFIZMAS
00345 // =====
00346
00347 TEST_CASE("Polimorfizmas", "[polimorfizmas]")
00348 {
00349     studentas s = kurkTestini();
00350     s.calculateGalutinis();
00351
00352     SECTION("virtualus metodai per bazines klases nuoroda") {
00353         zmogus& z = s;
00354         REQUIRE(z.tipas() == "Studentas");
00355         REQUIRE(z.galutinis() == Approx(8.0));
00356         REQUIRE(z vardas() == "Jonas");
00357         REQUIRE(z.pilnasVardas() == "Jonas Jonaitis");
00358     }
00359
00360     SECTION("virtualus metodai per unique_ptr") {
00361         std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00362             "Ona", "Onaite", Vector<int>{ 10, 10 }, 10);
00363         REQUIRE(p->tipas() == "Studentas");
00364         REQUIRE(p->pilnasVardas() == "Ona Onaite");
00365     }
00366
00367     SECTION("dynamic_cast atgal i isvestine klase") {
00368         std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00369             "Ona", "Onaite", Vector<int>{ 10, 10 }, 10);
00370         studentas* sp = dynamic_cast<studentas*>(p.get());
00371         REQUIRE(sp != nullptr);
00372         REQUIRE(sp->nd().size() == 2);
00373     }
00374
00375     SECTION("polimorfinis konteineris") {
00376         std::vector<std::unique_ptr<zmogus>> zmones;
00377         zmones.push_back(std::make_unique<studentas>(
00378             "Pirmas", "Pirmaitis", Vector<int>{ 10, 10 }, 10));
00379         zmones.push_back(std::make_unique<studentas>(
00380             "Antras", "Antraitis", Vector<int>{ 1, 1 }, 1));
00381
00382         for (auto& z : zmones)
00383             if (auto* sp = dynamic_cast<studentas*>(z.get()))
00384                 sp->calculateGalutinis();
00385
00386         REQUIRE(zmones[0]->galutinis() == Approx(10.0));
00387         REQUIRE(zmones[1]->galutinis() == Approx(1.0));
00388     }
00389 }

```



```

00390
00391 // =====
00392 // 5. ĮVESTIES / IŠVESTIES OPERATORIAI
00393 // =====
00394
00395 TEST_CASE("Ivesties operatorius », "[io]")
00396 {
00397     SECTION("nuskaito iš istringstream") {
00398         std::istringstream iss("Ona Onaitė 10 9 8 6");
00399         studentas s;
00400         iss » s;
00401         REQUIRE(s.vardas() == "Ona");
00402         REQUIRE(s.pavarde() == "Onaitė");
00403         REQUIRE(s.nd().size() == 3);
00404         REQUIRE(s.egzaminas() == 6);
00405     }
00406
00407     SECTION("polimorfinis » per zmogus nuoroda") {
00408         studentas s;
00409         zmogus& z = s;
00410         std::istringstream iss("Ona Onaitė 9 8 7");
00411         iss » z;
00412         REQUIRE(s.vardas() == "Ona");
00413         REQUIRE(s.egzaminas() == 7);
00414     }
00415 }
00416
00417 TEST_CASE("Išvesties operatorius «", "[io]")
00418 {
00419     studentas s = kurkTestini();
00420     s.calculateGalutinis();
00421
00422     SECTION("išveda vardą, pavardę ir balus") {
00423         std::ostringstream oss;
00424         oss « s;
00425         const std::string rez = oss.str();
00426         REQUIRE(rez.find("Jonas") != std::string::npos);
00427         REQUIRE(rez.find("Jonaitis") != std::string::npos);
00428         REQUIRE(rez.find("8.00") != std::string::npos);
00429     }
00430
00431     SECTION("polimorfinis « per zmogus nuoroda") {
00432         zmogus& z = s;
00433         std::ostringstream oss;
00434         oss « z;
00435         REQUIRE(oss.str().find("Jonas") != std::string::npos);
00436     }
00437 }
00438
00439 TEST_CASE("Failo I/O: parseFromLine ir appendListTo", "[io][failai]")
00440 {
00441     SECTION("parseFromLine nuskaito eilutę") {
00442         studentas s;
00443         s.parseFromLine("Antanas Antanaitis 7 8 9 10 5", 4, 1);
00444         REQUIRE(s.vardas() == "Antanas");
00445         REQUIRE(s.pavarde() == "Antanaitis");
00446         REQUIRE(s.nd().size() == 4);
00447         REQUIRE(s.egzaminas() == 5);
00448     }
00449
00450     SECTION("parseFromLine meta klaida blogam pazymiui") {
00451         studentas s;
00452         REQUIRE_THROWS_AS(s.parseFromLine("Vardas Pavarde 99 5", 2, 7),
00453             std::runtime_error);
00454     }
00455
00456     SECTION("parseFromLine meta klaida trukstant duomeniui") {
00457         studentas s;
00458         REQUIRE_THROWS_AS(s.parseFromLine("TikVardas", 2, 3),
00459             std::runtime_error);
00460     }
00461
00462     SECTION("appendListTo suformatuoja eilutę") {
00463         studentas s("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00464         std::string buferis;
00465         s.appendListTo(buferis);
00466         REQUIRE(buferis.find("Rasa") != std::string::npos);
00467         REQUIRE(buferis.back() == '\\n');
00468         REQUIRE(buferis.size() == 25 + 25 + 4 * 10 + 1);
00469     }
00470
00471     SECTION("round-trip: išvesta eilutė nuskaitoma atgal") {
00472         // Šis testas garantuoja, kad writeStudentaiToFile sukurta
00473         // faila teisingai nuskaitys readStudentaiFromFile
00474         studentas originalas("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00475         std::string buferis;
00476         originalas.appendListTo(buferis);

```

```

00477
00478     studentas nuskaitytas;
00479     nuskaitytas.parseFromLine(buferis, 3, 1);
00480
00481     REQUIRE(nuskaitytas.vardas() == originalas.vardas());
00482     REQUIRE(nuskaitytas.pavarde() == originalas.pavarde());
00483     REQUIRE(nuskaitytas.nd() == originalas.nd());
00484     REQUIRE(nuskaitytas.egzaminas() == originalas.egzaminas());
00485     REQUIRE(nuskaitytas == originalas);
00486 }
00487 }
00488
00489 // =====
00490 // 6. PERDENGTI OPERATORIAI
00491 // =====
00492
00493 TEST_CASE("Lyginimo operatoriai", "[operatoriai]")
00494 {
00495     studentas a("Jonas", "Jonaitis", { 5, 6 }, 7);
00496     studentas b("Jonas", "Jonaitis", { 5, 6 }, 7);
00497     studentas c("Petras", "Petraitis", { 5, 6 }, 7);
00498
00499     SECTION("== ir !=") {
00500         REQUIRE(a == b);
00501         REQUIRE(a != c);
00502         REQUIRE_FALSE(a == c);
00503     }
00504
00505     SECTION("< ir > rikiuoja pagal pavarde") {
00506         studentas x("Jonas", "Adamkus", { 5 }, 7);
00507         studentas y("Jonas", "Zukauskas", { 5 }, 7);
00508         REQUIRE(x < y);
00509         REQUIRE(y > x);
00510     }
00511
00512     SECTION("esant vienodoms pavardems lyginama pagal varda") {
00513         studentas x("Antanas", "Petraitis", { 5 }, 7);
00514         studentas y("Zigmas", "Petraitis", { 5 }, 7);
00515         REQUIRE(x < y);
00516     }
00517
00518     SECTION("veikia su std::sort") {
00519         std::vector<studentas> v = { c, a };
00520         std::sort(v.begin(), v.end());
00521         REQUIRE(v[0].pavarde() == "Jonaitis");
00522     }
00523 }
00524
00525 TEST_CASE("Indeksavimo operatorius []", "[operatoriai]")
00526 {
00527     studentas s("Jonas", "Jonaitis", { 3, 6, 9 }, 10);
00528
00529     SECTION("grazina teisinga pazymi") {
00530         REQUIRE(s[0] == 3);
00531         REQUIRE(s[1] == 6);
00532         REQUIRE(s[2] == 9);
00533     }
00534
00535     SECTION("meta out_of_range uz ribu") {
00536         REQUIRE_THROWS_AS(s[3], std::out_of_range);
00537         REQUIRE_THROWS_AS(s[99], std::out_of_range);
00538     }
00539 }
00540
00541 // =====
00542 // 7. SKAIČIAVIMO LOGIKA
00543 // =====
00544
00545 TEST_CASE("calculateGalutinis", "[skaiciavimas]")
00546 {
00547     SECTION("nelyginis ND kiekis") {
00548         // ND: 2,4,6,8,10 → vidurkis 6.0, mediana 6.0
00549         studentas s("Testas", "Testaitis", { 2, 4, 6, 8, 10 }, 5);
00550         s.calculateGalutinis();
00551         REQUIRE(s.galutinisVid() == Approx(0.4 * 6.0 + 0.6 * 5));
00552         REQUIRE(s.galutinisMed() == Approx(0.4 * 6.0 + 0.6 * 5));
00553     }
00554
00555     SECTION("lyginis ND kiekis -- mediana yra dviejų vidurio reikšmių vidurkis") {
00556         // ND: 1,2,3,4 → vidurkis 2.5, mediana (2+3)/2 = 2.5
00557         studentas s("Testas", "Testaitis", { 1, 2, 3, 4 }, 10);
00558         s.calculateGalutinis();
00559         REQUIRE(s.galutinisMed() == Approx(0.4 * 2.5 + 0.6 * 10));
00560     }
00561
00562     SECTION("vidurkis ir mediana skiriasi esant issibarsčiusiems duomenims") {
00563         // ND: 1,1,1,1,10 → vidurkis 2.8, mediana 1.0

```

```

00564         studentas s("Testas", "Testaitis", { 1, 1, 1, 1, 10 }, 5);
00565         s.calculateGalutinis();
00566         REQUIRE(s.galutinisVid() == Approx(0.4 * 2.8 + 0.6 * 5));
00567         REQUIRE(s.galutinisMed() == Approx(0.4 * 1.0 + 0.6 * 5));
00568         REQUIRE(s.galutinisVid() != Approx(s.galutinisMed()));
00569     }
00570
00571     SECTION("tuscias ND sarasas") {
00572         studentas s;
00573         s.setVardas("Tuscias");
00574         s.setPavarde("Tusciaitis");
00575         s.setEgzaminas(8);
00576         s.calculateGalutinis();
00577         REQUIRE(s.galutinisVid() == Approx(0.6 * 8));
00578         REQUIRE(s.galutinisMed() == Approx(0.6 * 8));
00579     }
00580
00581     SECTION("nesurikiuoti ND -- mediana skaiciuojama teisingai") {
00582         studentas s("Testas", "Testaitis", { 9, 1, 5 }, 10);
00583         s.calculateGalutinis();
00584         // Surikiavus: 1,5,9 + mediana 5
00585         REQUIRE(s.galutinisMed() == Approx(0.4 * 5.0 + 0.6 * 10));
00586     }
00587 }
00588
00589 // =====
00590 // 8. SET'ERIAI
00591 // =====
00592
00593 TEST_CASE("Seteriai", "[seteriai]")
00594 {
00595     studentas s;
00596
00597     SECTION("paveldeti bazines klases seteriai") {
00598         s.setVardas("Naujas");
00599         s.setPavarde("Naujaitis");
00600         REQUIRE(s.vardas() == "Naujas");
00601         REQUIRE(s.pavarde() == "Naujaitis");
00602     }
00603
00604     SECTION("bazines klases seteriai validuoja") {
00605         REQUIRE_THROWS_AS(s.setVardas(""), std::invalid_argument);
00606         REQUIRE_THROWS_AS(s.setPavarde(""), std::invalid_argument);
00607     }
00608
00609     SECTION("isvestines klases seteriai") {
00610         s.setNd({ 5, 6, 7 });
00611         s.setEgzaminas(9);
00612         REQUIRE(s.nd().size() == 3);
00613         REQUIRE(s.egzaminas() == 9);
00614
00615         s.addPazymys(10);
00616         REQUIRE(s.nd().size() == 4);
00617         REQUIRE(s.nd().back() == 10);
00618     }
00619
00620     SECTION("isvestines klases seteriai validuoja") {
00621         REQUIRE_THROWS_AS(s.setEgzaminas(50), std::runtime_error);
00622         REQUIRE_THROWS_AS(s.setNd({ 5, 99 }), std::runtime_error);
00623         REQUIRE_THROWS_AS(s.addPazymys(0), std::runtime_error);
00624     }
00625
00626     SECTION("isvalyk istustina abieju klasiu laukus") {
00627         studentas t = kurkTestini();
00628         t.calculateGalutinis();
00629         t.isvalyk();
00630         REQUIRE(t.vardas().empty());
00631         REQUIRE(t.pavarde().empty());
00632         REQUIRE(t.nd().empty());
00633         REQUIRE(t.egzaminas() == 0);
00634         REQUIRE(t.galutinisVid() == Approx(0.0));
00635     }
00636 }
00637
00638 // =====
00639 // 9. VEIKIMAS KONTEINERIUOSE
00640 // =====
00641
00642 TEST_CASE("Veikimas su STL konteineriais", "[konteineriai]")
00643 {
00644     SECTION("vector perskirstymas islaiko duomenis") {
00645         std::vector<studentas> v;
00646         for (int i = 0; i < 4; ++i)
00647             v.push_back(studentas("Vardas" + std::to_string(i),
00648                                   "Pavarde" + std::to_string(i),
00649                                   { 5, 6, 7 }, 8));
00650         v.reserve(100); // priverstinis perskirstymas
    }

```

```

00651         REQUIRE(v.size() == 4);
00652         REQUIRE(v[0].vardas() == "Vardas0");
00653         REQUIRE(v[3].vardas() == "Vardas3");
00654     }
00655
00656     SECTION("vektorius perkeltas su std::move") {
00657         std::vector<studentas> v;
00658         v.push_back(kurkTestini());
00659         v.push_back(kurkTestini());
00660
00661         std::vector<studentas> v2 = std::move(v);
00662         REQUIRE(v2.size() == 2);
00663         REQUIRE(v2[0].vardas() == "Jonas");
00664     }
00665
00666     SECTION("veikia su std::list ir std::deque") {
00667         std::list<studentas> l;
00668         std::deque<studentas> d;
00669         l.push_back(kurkTestini());
00670         d.push_back(kurkTestini());
00671         REQUIRE(l.front().vardas() == "Jonas");
00672         REQUIRE(d.front().vardas() == "Jonas");
00673     }
00674 }

```

5.6 tests/Vectortests.cpp Failo Nuoroda

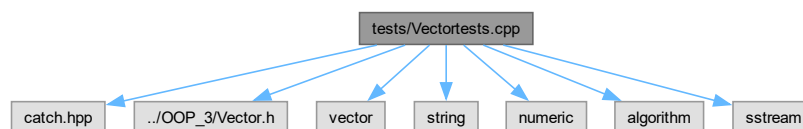
Unit testai nuosavai Vector klasei (Catch2).

```

#include "catch.hpp"
#include "../OOP_3/Vector.h"
#include <vector>
#include <string>
#include <numeric>
#include <algorithm>
#include <sstream>

```

Įtraukimo priklausomybių diagrama Vectortests.cpp:



Apibrėžimai

- `#define CATCH_CONFIG_MAIN`

Funkcijos

- `TEST_CASE` ("Konstruktoriai", "[konstruktoriai]")
- `TEST_CASE` ("Priskyrimas", "[priskyrimas]")
- `TEST_CASE` ("Elementu prieiga", "[prieiga]")
- `TEST_CASE` ("Iteratoriai", "[iteratoriai]")
- `TEST_CASE` ("Talpa", "[talpa]")
- `TEST_CASE` ("push_back ir pop_back", "[modifikavimas]")
- `TEST_CASE` ("emplace_back", "[modifikavimas]")
- `TEST_CASE` ("insert", "[modifikavimas]")
- `TEST_CASE` ("erase", "[modifikavimas]")
- `TEST_CASE` ("clear ir resize", "[modifikavimas]")
- `TEST_CASE` ("swap", "[modifikavimas]")
- `TEST_CASE` ("Lyginimo operatoriai", "[operatoriai]")

5.6.1 Smulkus aprašymas

Unit testai nuosavai Vector klasei (Catch2).

Kiekvienam testui taikomas tas pats principas: ta pati operacija atliekama su `std::vector` ir su `Vector`, o rezultatai lyginami. Taip tikrinama ne tik ar `Vector` veikia, bet ar veikia **taip pat** kaip standartinis konteineris.

Paleidimas: `./vectorTests` — visi testai `./vectorTests "[talpa]"` — tik talpos testai `./vectorTests -l` — testų sąrašas

Apibrėžimas faile [Vectortests.cpp](#).

5.6.2 Apibrėžimų Dokumentacija

5.6.2.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 16.

5.6.3 Funkcijos Dokumentacija

5.6.3.1 TEST_CASE() [1/12]

```
TEST_CASE (
    "Konstruktoriai" ,
    "" [konstruktoriai])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 73.

5.6.3.2 TEST_CASE() [2/12]

```
TEST_CASE (
    "Priskyrimas" ,
    "" [priskyrimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 157.

5.6.3.3 TEST_CASE() [3/12]

```
TEST_CASE (
    "Elementu prieiga" ,
    "" [prieiga])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 209.

5.6.3.4 TEST_CASE() [4/12]

```
TEST_CASE (
    "Iteratoriai" ,
    "" [iteratoriai])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 259.

5.6.3.5 TEST_CASE() [5/12]

```
TEST_CASE (
    "Talpa" ,
    "" [talpa])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 306.

5.6.3.6 TEST_CASE() [6/12]

```
TEST_CASE (
    "push_back ir pop_back" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 403.

5.6.3.7 TEST_CASE() [7/12]

```
TEST_CASE (
    "emplace_back" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 438.

5.6.3.8 TEST_CASE() [8/12]

```
TEST_CASE (
    "insert" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 463.

5.6.3.9 TEST_CASE() [9/12]

```
TEST_CASE (
    "erase" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 521.

5.6.3.10 TEST_CASE() [10/12]

```
TEST_CASE (
    "clear ir resize" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 555.

5.6.3.11 TEST_CASE() [11/12]

```
TEST_CASE (
    "swap" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 600.

5.6.3.12 TEST_CASE() [12/12]

```
TEST_CASE (
    "Lyginimo operatoriai" ,
    "" [operatoriai])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 629.

5.7 Vectortests.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file vectorTests.cpp
00003  * @brief Unit testai nuosavai Vector klasei (Catch2).
00004  *
00005  * Kiekvienam testui taikomas tas pats principas: ta pati operacija
00006  * atliekama su `std::vector` ir su `Vector`, o rezultatai lyginami.
00007  * Taip tikrinama ne tik ar `Vector` veikia, bet ar veikia **taip pat**
00008  * kaip standartinis konteineris.
00009  *
00010  * Paleidimas:
00011  *      ./vectorTests                -- visi testai
00012  *      ./vectorTests "[talpa]"      -- tik talpos testai
00013  *      ./vectorTests -l              -- testų sąrašas
00014  */
00015
00016 #define CATCH_CONFIG_MAIN
00017 #include "catch.hpp"
00018 #include "../OOP_3/Vector.h"
00019
00020 #include <vector>
```

```

00021 #include <string>
00022 #include <numeric>
00023 #include <algorithm>
00024 #include <sstream>
00025
00026 // =====
00027 // Pagalbinė klasė objektų gyvavimui seksti
00028 // =====
00029
00030 namespace {
00031
00032 /**
00033  * @brief Testinė klasė, skaičiuojanti konstruktorių ir destruktorių kvietimus.
00034  *
00035  * Leidžia patikrinti, ar Vector teisingai konstruoja ir naikina
00036  * elementus -- ypač svarbu perskirstymo metu.
00037  */
00038 struct Sekiklis {
00039     static int gyvu;
00040     static int kopiju;
00041     static int perkelimu;
00042     int reiksme;
00043
00044     Sekiklis(int v = 0) : reiksme(v) { ++gyvu; }
00045     Sekiklis(const Sekiklis& o) : reiksme(o.reiksme) { ++gyvu; ++kopiju; }
00046     Sekiklis(Sekiklis&& o) noexcept : reiksme(o.reiksme) { ++gyvu; ++perkelimu; }
00047     Sekiklis& operator=(const Sekiklis& o) { reiksme = o.reiksme; ++kopiju; return *this; }
00048     Sekiklis& operator=(Sekiklis&& o) noexcept { reiksme = o.reiksme; ++perkelimu; return *this; }
00049     ~Sekiklis() { --gyvu; }
00050
00051     bool operator==(const Sekiklis& o) const { return reiksme == o.reiksme; }
00052
00053     static void atstatyk() { gyvu = 0; kopiju = 0; perkelimu = 0; }
00054 };
00055
00056 int Sekiklis::gyvu = 0;
00057 int Sekiklis::kopiju = 0;
00058 int Sekiklis::perkelimu = 0;
00059
00060 /// Palygina Vector ir std::vector turinį
00061 template <typename T>
00062 bool sutampa(const Vector<T>& v, const std::vector<T>& sv) {
00063     return v.size() == sv.size()
00064         && std::equal(v.begin(), v.end(), sv.begin());
00065 }
00066
00067 } // anoniminis namespace
00068
00069 // =====
00070 // 1. KONSTRUKTORIAI
00071 // =====
00072
00073 TEST_CASE("Konstruktoriai", "[konstruktoriai]")
00074 {
00075     SECTION("numatytasis -- tuscias vektorius") {
00076         Vector<int> v;
00077         std::vector<int> sv;
00078         REQUIRE(v.size() == sv.size());
00079         REQUIRE(v.capacity() == sv.capacity());
00080         REQUIRE(v.empty() == sv.empty());
00081         REQUIRE(v.data() == nullptr);
00082     }
00083
00084     SECTION("n elementu su reiksme") {
00085         Vector<int> v(5, 42);
00086         std::vector<int> sv(5, 42);
00087         REQUIRE(sutampa(v, sv));
00088         REQUIRE(v.size() == 5);
00089         REQUIRE(v[0] == 42);
00090         REQUIRE(v[4] == 42);
00091     }
00092
00093     SECTION("n numatytai sukonstruotu elementu") {
00094         Vector<int> v(5);
00095         std::vector<int> sv(5);
00096         REQUIRE(sutampa(v, sv));
00097         REQUIRE(v[0] == 0); // int() = 0
00098     }
00099
00100     SECTION("is iteratoriu intervalu") {
00101         std::vector<int> saltinis = { 1, 2, 3, 4, 5 };
00102         Vector<int> v(saltinis.begin(), saltinis.end());
00103         REQUIRE(sutampa(v, saltinis));
00104     }
00105
00106     SECTION("is initializer_list") {
00107         Vector<int> v = { 1, 2, 3 };

```

```

00108         std::vector<int> sv = { 1, 2, 3 };
00109         REQUIRE(sutampa(v, sv));
00110     }
00111
00112     SECTION("Vector(5, 10) NEinterpretuojamas kaip intervalas") {
00113         // Be enable_if sios eilutes butu klaidingai parinktas
00114         // intervalo konstruktorius
00115         Vector<int> v(5, 10);
00116         REQUIRE(v.size() == 5);
00117         REQUIRE(v[0] == 10);
00118     }
00119
00120     SECTION("kopijavimo konstruktorius -- gili kopija") {
00121         Vector<int> originalas = { 1, 2, 3 };
00122         Vector<int> kopija(originalas);
00123
00124         REQUIRE(kopija == originalas);
00125         REQUIRE(kopija.data() != originalas.data()); // skirtinga atmintis
00126
00127         kopija[0] = 99;
00128         REQUIRE(originalas[0] == 1); // originalas nepakito
00129     }
00130
00131     SECTION("perkelimo konstruktorius") {
00132         Vector<int> saltinis = { 1, 2, 3 };
00133         const int* senasAdresas = saltinis.data();
00134
00135         Vector<int> tikslas(std::move(saltinis));
00136
00137         REQUIRE(tikslas.size() == 3);
00138         REQUIRE(tikslas.data() == senasAdresas); // ta pati atmintis
00139         REQUIRE(saltinis.size() == 0); // saltinis tuscias
00140         REQUIRE(saltinis.data() == nullptr);
00141     }
00142
00143     SECTION("konstruktoriai teisingai naikina objektus") {
00144         Sekiklis::atstatyk();
00145         {
00146             Vector<Sekiklis> v(10);
00147             REQUIRE(Sekiklis::gyvu == 10);
00148         }
00149         REQUIRE(Sekiklis::gyvu == 0);
00150     }
00151 }
00152
00153 // =====
00154 // 2. PRISKYRIMAS
00155 // =====
00156
00157 TEST_CASE("Priskyrimas", "[priskyrimas]")
00158 {
00159     SECTION("kopijavimo priskyrimas") {
00160         Vector<int> a = { 1, 2, 3 };
00161         Vector<int> b;
00162         b = a;
00163         REQUIRE(b == a);
00164         REQUIRE(b.data() != a.data());
00165     }
00166
00167     SECTION("priskyrimas sau nesugadina") {
00168         Vector<int> a = { 1, 2, 3 };
00169         a = a;
00170         REQUIRE(a.size() == 3);
00171         REQUIRE(a[0] == 1);
00172     }
00173
00174     SECTION("perkelimo priskyrimas") {
00175         Vector<int> a = { 1, 2, 3 };
00176         Vector<int> b = { 9 };
00177         b = std::move(a);
00178         REQUIRE(b.size() == 3);
00179         REQUIRE(a.size() == 0);
00180     }
00181
00182     SECTION("priskyrimas is initializer_list") {
00183         Vector<int> v;
00184         v = { 4, 5, 6 };
00185         REQUIRE(v.size() == 3);
00186         REQUIRE(v[1] == 5);
00187     }
00188
00189     SECTION("assign su n kopiju") {
00190         Vector<int> v = { 1, 2, 3 };
00191         std::vector<int> sv = { 1, 2, 3 };
00192         v.assign(5, 7);
00193         sv.assign(5, 7);
00194         REQUIRE(sutampa(v, sv));

```



```

00195     }
00196
00197     SECTION("assign su intervalu") {
00198         std::vector<int> saltinis = { 8, 9 };
00199         Vector<int> v = { 1, 2, 3 };
00200         v.assign(saltinis.begin(), saltinis.end());
00201         REQUIRE(sutampa(v, saltinis));
00202     }
00203 }
00204
00205 // =====
00206 // 3. ELEMENTŲ PRIEIGA
00207 // =====
00208
00209 TEST_CASE("Elementu prieiga", "[prieiga]")
00210 {
00211     Vector<int> v = { 10, 20, 30, 40 };
00212     std::vector<int> sv = { 10, 20, 30, 40 };
00213
00214     SECTION("operator[]") {
00215         REQUIRE(v[0] == sv[0]);
00216         REQUIRE(v[3] == sv[3]);
00217         v[1] = 99;
00218         REQUIRE(v[1] == 99);
00219     }
00220
00221     SECTION("at() su ribu tikrinimu") {
00222         REQUIRE(v.at(2) == sv.at(2));
00223         REQUIRE_THROWS_AS(v.at(4), std::out_of_range);
00224         REQUIRE_THROWS_AS(v.at(100), std::out_of_range);
00225
00226         // std::vector meta ta pacia isimti
00227         REQUIRE_THROWS_AS(sv.at(4), std::out_of_range);
00228     }
00229
00230     SECTION("front ir back") {
00231         REQUIRE(v.front() == sv.front());
00232         REQUIRE(v.back() == sv.back());
00233         v.front() = 1;
00234         v.back() = 4;
00235         REQUIRE(v[0] == 1);
00236         REQUIRE(v[3] == 4);
00237     }
00238
00239     SECTION("data() grazina rodykle i masyva") {
00240         int* p = v.data();
00241         REQUIRE(p[0] == 10);
00242         REQUIRE(p[3] == 40);
00243     }
00244
00245     SECTION("const prieiga") {
00246         const Vector<int>& cv = v;
00247         REQUIRE(cv[0] == 10);
00248         REQUIRE(cv.at(1) == 20);
00249         REQUIRE(cv.front() == 10);
00250         REQUIRE(cv.back() == 40);
00251         REQUIRE(cv.data() != nullptr);
00252     }
00253 }
00254
00255 // =====
00256 // 4. ITERATORIAI
00257 // =====
00258
00259 TEST_CASE("Iteratoriai", "[iteratoriai]")
00260 {
00261     Vector<int> v = { 1, 2, 3, 4, 5 };
00262     std::vector<int> sv = { 1, 2, 3, 4, 5 };
00263
00264     SECTION("begin ir end") {
00265         REQUIRE(*v.begin() == 1);
00266         REQUIRE(*(v.end() - 1) == 5);
00267         REQUIRE(v.end() - v.begin() == 5);
00268     }
00269
00270     SECTION("range-based for") {
00271         int suma = 0;
00272         for (int x : v) suma += x;
00273         REQUIRE(suma == 15);
00274     }
00275
00276     SECTION("atvirkstiniai iteratoriai") {
00277         std::vector<int> atvirksciai;
00278         for (auto it = v.rbegin(); it != v.rend(); ++it)
00279             atvirksciai.push_back(*it);
00280         REQUIRE(atvirksciai == std::vector<int>{ 5, 4, 3, 2, 1 });
00281     }

```

```

00282
00283 SECTION("const iteratoriai") {
00284     const Vector<int>& cv = v;
00285     REQUIRE(*cv.cbegin() == 1);
00286     REQUIRE(*(cv.cend() - 1) == 5);
00287     REQUIRE(*cv.crbegin() == 5);
00288 }
00289
00290 SECTION("veikia su STL algoritmais") {
00291     REQUIRE(std::accumulate(v.begin(), v.end(), 0) == 15);
00292     REQUIRE(*std::max_element(v.begin(), v.end()) == 5);
00293     REQUIRE(std::count(v.begin(), v.end(), 3) == 1);
00294
00295     std::sort(v.begin(), v.end(), std::greater<int>());
00296     REQUIRE(v[0] == 5);
00297
00298     REQUIRE(std::find(v.begin(), v.end(), 3) != v.end());
00299 }
00300 }
00301
00302 // =====
00303 // 5. TALPA -- svarbiausias skyrius
00304 // =====
00305
00306 TEST_CASE("Talpa", "[talpa]")
00307 {
00308     SECTION("empty ir size") {
00309         Vector<int> v;
00310         REQUIRE(v.empty());
00311         REQUIRE(v.size() == 0);
00312
00313         v.push_back(1);
00314         REQUIRE_FALSE(v.empty());
00315         REQUIRE(v.size() == 1);
00316     }
00317
00318     SECTION("capacity dvigubinama augant") {
00319         Vector<int> v;
00320         REQUIRE(v.capacity() == 0);
00321
00322         v.push_back(1);
00323         REQUIRE(v.capacity() == 1);
00324
00325         v.push_back(2);
00326         REQUIRE(v.capacity() == 2);
00327
00328         v.push_back(3);
00329         REQUIRE(v.capacity() == 4);
00330
00331         v.push_back(4);
00332         REQUIRE(v.capacity() == 4);
00333
00334         v.push_back(5);
00335         REQUIRE(v.capacity() == 8);
00336     }
00337
00338     SECTION("reserve paskiria atminti, bet nekuria elementu") {
00339         Vector<int> v;
00340         v.reserve(100);
00341         REQUIRE(v.capacity() >= 100);
00342         REQUIRE(v.size() == 0);
00343         REQUIRE(v.empty());
00344     }
00345
00346     SECTION("reserve nemazina talpos") {
00347         Vector<int> v;
00348         v.reserve(100);
00349         v.reserve(10);
00350         REQUIRE(v.capacity() >= 100);
00351     }
00352
00353     SECTION("reserve panaikina persikirstymus") {
00354         Vector<int> v;
00355         v.reserve(1000);
00356         const int* pries = v.data();
00357
00358         for (int i = 0; i < 1000; ++i)
00359             v.push_back(i);
00360
00361         REQUIRE(v.data() == pries); // atmintis nepersikele
00362     }
00363
00364     SECTION("shrink_to_fit atlaisvina nepanaudota atminti") {
00365         Vector<int> v;
00366         v.reserve(100);
00367         v.push_back(1);
00368         v.push_back(2);

```

```

00369         REQUIRE(v.capacity() >= 100);
00370
00371         v.shrink_to_fit();
00372         REQUIRE(v.capacity() == 2);
00373         REQUIRE(v.size() == 2);
00374         REQUIRE(v[0] == 1);
00375     }
00376
00377     SECTION("max_size grazina teigiama reiksme") {
00378         Vector<int> v;
00379         REQUIRE(v.max_size() > 0);
00380     }
00381
00382     SECTION("perskirstymu skaicius atitinka log2(n)") {
00383         Vector<int> v;
00384         int perskirstymu = 0;
00385         size_t senaTalpa = v.capacity();
00386
00387         for (int i = 0; i < 1000; ++i) {
00388             v.push_back(i);
00389             if (v.capacity() != senaTalpa) {
00390                 ++perskirstymu;
00391                 senaTalpa = v.capacity();
00392             }
00393         }
00394         // 1000 elementu + talpa 1,2,4,...,1024 + 11 perskirstymu
00395         REQUIRE(perskirstymu == 11);
00396     }
00397 }
00398
00399 // =====
00400 // 6. MODIFIKAVIMAS
00401 // =====
00402
00403 TEST_CASE("push_back ir pop_back", "[modifikavimas]")
00404 {
00405     SECTION("push_back prideda i gala") {
00406         Vector<int> v;
00407         std::vector<int> sv;
00408         for (int i = 1; i <= 10; ++i) { v.push_back(i); sv.push_back(i); }
00409         REQUIRE(sutampa(v, sv));
00410     }
00411
00412     SECTION("push_back su perkelimu") {
00413         Vector<std::string> v;
00414         std::string s = "labas";
00415         v.push_back(std::move(s));
00416         REQUIRE(v[0] == "labas");
00417         REQUIRE(s.empty()); // perkelta
00418     }
00419
00420     SECTION("pop_back pasalina paskutini") {
00421         Vector<int> v = { 1, 2, 3 };
00422         v.pop_back();
00423         REQUIRE(v.size() == 2);
00424         REQUIRE(v.back() == 2);
00425     }
00426
00427     SECTION("pop_back sunaikina objekta") {
00428         Sekiklis::atstatyk();
00429         Vector<Sekiklis> v;
00430         v.push_back(Sekiklis(1));
00431         v.push_back(Sekiklis(2));
00432         const int pries = Sekiklis::gyvu;
00433         v.pop_back();
00434         REQUIRE(Sekiklis::gyvu == pries - 1);
00435     }
00436 }
00437
00438 TEST_CASE("emplace_back", "[modifikavimas]")
00439 {
00440     SECTION("sukuria objekta vietoje") {
00441         Vector<std::string> v;
00442         v.emplace_back(5, 'a'); // std::string(5, 'a')
00443         REQUIRE(v[0] == "aaaaa");
00444     }
00445
00446     SECTION("grazina nuoroda i sukurta elementa") {
00447         Vector<int> v;
00448         int& r = v.emplace_back(42);
00449         REQUIRE(r == 42);
00450         r = 99;
00451         REQUIRE(v[0] == 99);
00452     }
00453
00454     SECTION("emplace_back nekopijuoja") {
00455         Sekiklis::atstatyk();

```

```

00456         Vector<Sekiklis> v;
00457         v.reserve(10);
00458         v.emplace_back(5);
00459         REQUIRE(Sekiklis::kopiju == 0);    // jokiu kopiju
00460     }
00461 }
00462
00463 TEST_CASE("insert", "[modifikavimas]")
00464 {
00465     SECTION("insert i pradzia") {
00466         Vector<int> v = { 2, 3 };
00467         std::vector<int> sv = { 2, 3 };
00468         v.insert(v.begin(), 1);
00469         sv.insert(sv.begin(), 1);
00470         REQUIRE(sutampa(v, sv));
00471     }
00472
00473     SECTION("insert i vidury") {
00474         Vector<int> v = { 1, 3 };
00475         std::vector<int> sv = { 1, 3 };
00476         v.insert(v.begin() + 1, 2);
00477         sv.insert(sv.begin() + 1, 2);
00478         REQUIRE(sutampa(v, sv));
00479     }
00480
00481     SECTION("insert i gala") {
00482         Vector<int> v = { 1, 2 };
00483         v.insert(v.end(), 3);
00484         REQUIRE(v.size() == 3);
00485         REQUIRE(v.back() == 3);
00486     }
00487
00488     SECTION("insert n kopiju") {
00489         Vector<int> v = { 1, 5 };
00490         std::vector<int> sv = { 1, 5 };
00491         v.insert(v.begin() + 1, 3, 9);
00492         sv.insert(sv.begin() + 1, 3, 9);
00493         REQUIRE(sutampa(v, sv));
00494     }
00495
00496     SECTION("insert intervala") {
00497         std::vector<int> saltinis = { 2, 3, 4 };
00498         Vector<int> v = { 1, 5 };
00499         std::vector<int> sv = { 1, 5 };
00500         v.insert(v.begin() + 1, saltinis.begin(), saltinis.end());
00501         sv.insert(sv.begin() + 1, saltinis.begin(), saltinis.end());
00502         REQUIRE(sutampa(v, sv));
00503     }
00504
00505     SECTION("insert initializer_list") {
00506         Vector<int> v = { 1, 4 };
00507         v.insert(v.begin() + 1, { 2, 3 });
00508         REQUIRE(v.size() == 4);
00509         REQUIRE(v[1] == 2);
00510         REQUIRE(v[2] == 3);
00511     }
00512
00513     SECTION("insert grazina iteratoriu i iterpta elementa") {
00514         Vector<int> v = { 1, 3 };
00515         auto it = v.insert(v.begin() + 1, 2);
00516         REQUIRE(*it == 2);
00517         REQUIRE(it == v.begin() + 1);
00518     }
00519 }
00520
00521 TEST_CASE("erase", "[modifikavimas]")
00522 {
00523     SECTION("erase viena elementa") {
00524         Vector<int> v = { 1, 2, 3, 4 };
00525         std::vector<int> sv = { 1, 2, 3, 4 };
00526         v.erase(v.begin() + 1);
00527         sv.erase(sv.begin() + 1);
00528         REQUIRE(sutampa(v, sv));
00529     }
00530
00531     SECTION("erase intervala") {
00532         Vector<int> v = { 1, 2, 3, 4, 5 };
00533         std::vector<int> sv = { 1, 2, 3, 4, 5 };
00534         v.erase(v.begin() + 1, v.begin() + 4);
00535         sv.erase(sv.begin() + 1, sv.begin() + 4);
00536         REQUIRE(sutampa(v, sv));
00537     }
00538
00539     SECTION("erase grazina iteratoriu i kita elementa") {
00540         Vector<int> v = { 1, 2, 3 };
00541         auto it = v.erase(v.begin());
00542         REQUIRE(*it == 2);

```

```

00543     }
00544
00545     SECTION("erase sunaikina objektus") {
00546         Sekiklis::atstatyk();
00547         Vector<Sekiklis> v;
00548         for (int i = 0; i < 5; ++i) v.emplace_back(i);
00549         const int pries = Sekiklis::gyvu;
00550         v.erase(v.begin(), v.begin() + 2);
00551         REQUIRE(Sekiklis::gyvu == pries - 2);
00552     }
00553 }
00554
00555 TEST_CASE("clear ir resize", "[modifikavimas]")
00556 {
00557     SECTION("clear pasalina elementus, bet ne talpa") {
00558         Vector<int> v = { 1, 2, 3 };
00559         const size_t talpa = v.capacity();
00560         v.clear();
00561         REQUIRE(v.size() == 0);
00562         REQUIRE(v.empty());
00563         REQUIRE(v.capacity() == talpa);    // talpa nepakito
00564     }
00565
00566     SECTION("clear sunaikina objektus") {
00567         Sekiklis::atstatyk();
00568         Vector<Sekiklis> v;
00569         for (int i = 0; i < 5; ++i) v.emplace_back(i);
00570         v.clear();
00571         REQUIRE(Sekiklis::gyvu == 0);
00572     }
00573
00574     SECTION("resize didinant") {
00575         Vector<int> v = { 1, 2 };
00576         std::vector<int> sv = { 1, 2 };
00577         v.resize(5);
00578         sv.resize(5);
00579         REQUIRE(sutampa(v, sv));
00580         REQUIRE(v[4] == 0);
00581     }
00582
00583     SECTION("resize didinant su reiksme") {
00584         Vector<int> v = { 1, 2 };
00585         std::vector<int> sv = { 1, 2 };
00586         v.resize(5, 9);
00587         sv.resize(5, 9);
00588         REQUIRE(sutampa(v, sv));
00589         REQUIRE(v[4] == 9);
00590     }
00591
00592     SECTION("resize mazinant") {
00593         Vector<int> v = { 1, 2, 3, 4, 5 };
00594         v.resize(2);
00595         REQUIRE(v.size() == 2);
00596         REQUIRE(v.back() == 2);
00597     }
00598 }
00599
00600 TEST_CASE("swap", "[modifikavimas]")
00601 {
00602     SECTION("nario swap") {
00603         Vector<int> a = { 1, 2 };
00604         Vector<int> b = { 3, 4, 5 };
00605         const int* aData = a.data();
00606         const int* bData = b.data();
00607
00608         a.swap(b);
00609
00610         REQUIRE(a.size() == 3);
00611         REQUIRE(b.size() == 2);
00612         REQUIRE(a.data() == bData);    // sukeistos rodykles, ne elementai
00613         REQUIRE(b.data() == aData);
00614     }
00615
00616     SECTION("ne-nare swap funkcija") {
00617         Vector<int> a = { 1 };
00618         Vector<int> b = { 2 };
00619         swap(a, b);
00620         REQUIRE(a[0] == 2);
00621         REQUIRE(b[0] == 1);
00622     }
00623 }
00624
00625 // =====
00626 // 7. NE-NARIAI OPERATORIAI
00627 // =====
00628
00629 TEST_CASE("Lyginimo operatoriai", "[operatoriai]")

```

```

00630 {
00631     Vector<int> a = { 1, 2, 3 };
00632     Vector<int> b = { 1, 2, 3 };
00633     Vector<int> c = { 1, 2, 4 };
00634     Vector<int> trumpas = { 1, 2 };
00635
00636     SECTION("== ir !=") {
00637         REQUIRE(a == b);
00638         REQUIRE(a != c);
00639         REQUIRE(a != trumpas);
00640     }
00641
00642     SECTION("leksikografinis palyginimas") {
00643         REQUIRE(a < c);
00644         REQUIRE(c > a);
00645         REQUIRE(trumpas < a);
00646         REQUIRE(a <= b);
00647         REQUIRE(a >= b);
00648     }
00649
00650     SECTION("rezultatai sutampa su std::vector") {
00651         std::vector<int> sa = { 1, 2, 3 };
00652         std::vector<int> sc = { 1, 2, 4 };
00653         REQUIRE((a < c) == (sa < sc));
00654         REQUIRE((a == b) == (sa == std::vector<int>{ 1, 2, 3 }));
00655     }
00656 }
00657
00658 TEST_CASE("erase ir erase_if (ne-nares)", "[operatoriai]")
00659 {
00660     SECTION("erase pasalina visas reikšmes kopijas") {
00661         Vector<int> v = { 1, 2, 3, 2, 5, 2 };
00662         const auto n = erase(v, 2);
00663         REQUIRE(n == 3);
00664         REQUIRE(v.size() == 3);
00665         REQUIRE(v == Vector<int>{ 1, 3, 5 });
00666     }
00667
00668     SECTION("erase_if pagal predikata") {
00669         Vector<int> v = { 1, 2, 3, 4, 5, 6 };
00670         const auto n = erase_if(v, [](int x) { return x % 2 == 0; });
00671         REQUIRE(n == 3);
00672         REQUIRE(v == Vector<int>{ 1, 3, 5 });
00673     }
00674 }
00675
00676 // =====
00677 // 8. SUDĖTINGI TIPAI
00678 // =====
00679
00680 TEST_CASE("Veikimas su sudetingais tipais", "[tipai]")
00681 {
00682     SECTION("Vector<std::string>") {
00683         Vector<std::string> v;
00684         v.push_back("pirmas");
00685         v.push_back("antras");
00686         v.emplace_back("trecias");
00687
00688         REQUIRE(v.size() == 3);
00689         REQUIRE(v[0] == "pirmas");
00690         REQUIRE(v.back() == "trecias");
00691     }
00692
00693     SECTION("Vector<Vector<int> -- vektorius vektoriuje") {
00694         Vector<Vector<int> matrica;
00695         matrica.push_back(Vector<int>{ 1, 2 });
00696         matrica.push_back(Vector<int>{ 3, 4 });
00697
00698         REQUIRE(matrica.size() == 2);
00699         REQUIRE(matrica[0][1] == 2);
00700         REQUIRE(matrica[1][0] == 3);
00701     }
00702
00703     SECTION("perskirstymas nesugadina objektu") {
00704         Sekiklis::atstatyk();
00705         {
00706             Vector<Sekiklis> v;
00707             for (int i = 0; i < 100; ++i)
00708                 v.emplace_back(i);
00709
00710             REQUIRE(Sekiklis::gyvu == 100);
00711             REQUIRE(v[0].reiksme == 0);
00712             REQUIRE(v[99].reiksme == 99);
00713         }
00714         REQUIRE(Sekiklis::gyvu == 0); // nera nutekejimo
00715     }
00716 }

```

```

00717     SECTION("perskirstymas naudoja move, ne copy") {
00718         Sekiklis::atstatyk();
00719         Vector<Sekiklis> v;
00720         v.reserve(2);
00721         v.emplace_back(1);
00722         v.emplace_back(2);
00723
00724         const int kopijuPries = Sekiklis::kopiju;
00725         v.emplace_back(3); // sukelia perskirstyma
00726
00727         // Sekiklis turi noexcept move, todel turi buti naudojamas
00728         REQUIRE(Sekiklis::kopiju == kopijuPries);
00729         REQUIRE(Sekiklis::perkelimu > 0);
00730     }
00731 }
00732
00733 // =====
00734 // 9. KRAŠTINIAI ATVEJAI
00735 // =====
00736
00737 TEST_CASE("Krastiniai atvejai", "[krastiniai]")
00738 {
00739     SECTION("operacijos su tusciu vektoriumi") {
00740         Vector<int> v;
00741         REQUIRE(v.begin() == v.end());
00742         REQUIRE(v.size() == 0);
00743         REQUIRE_NOTHROW(v.clear());
00744         REQUIRE_NOTHROW(v.shrink_to_fit());
00745         REQUIRE_THROWS_AS(v.at(0), std::out_of_range);
00746     }
00747
00748     SECTION("insert i tuscia vektoriu") {
00749         Vector<int> v;
00750         v.insert(v.begin(), 1);
00751         REQUIRE(v.size() == 1);
00752         REQUIRE(v[0] == 1);
00753     }
00754
00755     SECTION("erase visus elementus") {
00756         Vector<int> v = { 1, 2, 3 };
00757         v.erase(v.begin(), v.end());
00758         REQUIRE(v.empty());
00759     }
00760
00761     SECTION("insert nulis elementu") {
00762         Vector<int> v = { 1, 2 };
00763         v.insert(v.begin(), 0, 9);
00764         REQUIRE(v.size() == 2);
00765     }
00766
00767     SECTION("resize i ta pati dydi") {
00768         Vector<int> v = { 1, 2, 3 };
00769         v.resize(3);
00770         REQUIRE(v.size() == 3);
00771         REQUIRE(v[2] == 3);
00772     }
00773
00774     SECTION("didelis reserve") {
00775         Vector<int> v;
00776         REQUIRE_NOTHROW(v.reserve(100000));
00777         REQUIRE(v.capacity() >= 100000);
00778     }
00779 }

```

